

```

#!/usr/bin/env python3
from __future__ import annotations

# %% [markdown]
# # MuJoCo Admittance-Control Painting Demo
#
# This script is written in a notebook-friendly style so it can be copied into
# Google Colab or run directly as a `.py` file. It demonstrates a simplified
# 6-DOF "Piper-like" arm painting on a moving wall while regulating the normal
# contact force with a wall-referenced impedance-style controller.
#
# Key ideas:
# - The wall first moves toward the robot, then eases back while staying no farther
#   than its original `x` position along the surface normal.
# - The brush follows a nominal tangential painting path in the wall's `y-z` plane.
# - A wall-referenced impedance controller offsets the brush in `x` to regulate contact force.
# - If contact is lost, the controller performs a slow, safe search toward the wall.

# %% Imports and optional package installation
import importlib
import argparse
import json
import math
import os
import subprocess
import sys
import tempfile
import time
import xml.etree.ElementTree as ET
from pathlib import Path
from textwrap import dedent

SCRIPT_DIR = Path(__file__).resolve().parent if "__file__" in globals() else Path.cwd()
IN_COLAB = "google.colab" in sys.modules or os.getenv("COLAB_GPU") is not None
ALLOW_RUNTIME_PIP_INSTALL = IN_COLAB
FORCE_NONINTERACTIVE_MPL = sys.platform == "darwin" and not IN_COLAB

def choose_writable_dir(preferred_dir: Path, fallback_name: str) -> Path:
    """Return a writable directory, falling back to cwd or the system temp dir."""
    candidates = [
        preferred_dir,
        Path.cwd() / fallback_name,
        Path(tempfile.gettempdir()) / fallback_name,
    ]

    for candidate in candidates:
        try:
            candidate.mkdir(parents=True, exist_ok=True)
            if os.access(candidate, os.W_OK):
                return candidate
        except Exception:
            continue

    raise RuntimeError(f"Could not create a writable directory for {fallback_name}.")

MATPLOTLIB_CACHE_DIR = choose_writable_dir(SCRIPT_DIR / ".matplotlib-cache", ".matplotlib-cache")
os.environ.setdefault("MPLCONFIGDIR", str(MATPLOTLIB_CACHE_DIR))
if FORCE_NONINTERACTIVE_MPL:
    os.environ.setdefault("MPLBACKEND", "Agg")

def ensure_package(import_name: str, pip_name: str | None = None):
    """Import a package, installing it first if needed."""
    try:
        return importlib.import_module(import_name)
    except ImportError:
        install_name = pip_name or import_name
        if ALLOW_RUNTIME_PIP_INSTALL:

```

```
print(f"Installing missing package: {install_name}")
subprocess.check_call([sys.executable, "-m", "pip", "install", install_name])
return importlib.import_module(import_name)
```

```
raise RuntimeError(
    f"Missing Python package '{install_name}' in interpreter:\n"
    f" {sys.executable}\n\n"
    "This script does not auto-install packages in local desktop runs.\n"
    "In PyCharm, set the interpreter to the same Python that works in your terminal,\n"
    "for example `/opt/miniconda3/bin/python3`, then install:\n"
    " pip install mujoco numpy matplotlib imageio\n"
) from None
```

```
mujoco = ensure_package("mujoco")
np = ensure_package("numpy")
ensure_package("matplotlib")
plt = importlib.import_module("matplotlib.pyplot")
animation = importlib.import_module("matplotlib.animation")
imageio = ensure_package("imageio.v2", "imageio")
mujoco_viewer = importlib.import_module("mujoco.viewer")
try:
    imageio_ffmpeg = importlib.import_module("imageio_ffmpeg")
except ImportError:
    imageio_ffmpeg = None
```

```
try:
    ipython_display = importlib.import_module("IPython.display")
    IPyImage = ipython_display.Image
    display = ipython_display.display
except ImportError:
    IPyImage = None
    display = None
```

```
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

```
def set_vis_flag(flag_container, flag_name: str, enabled: bool):
    """Set a MuJoCo viewer flag only when the enum exists in the installed version."""
    vis_flag = getattr(mujoco.mjtVisFlag, flag_name, None)
    if vis_flag is not None:
        flag_container.flags[vis_flag] = enabled
```

```
# %% Tuning parameters
# Desired normal contact force at the brush tip [N].
F_DES = 10.0

# Impedance command smoothing for IK-friendly position commands.
IMP_XCMD_LPF_ALPHA = 0.35
IMP_MAX_XCMD_RATE = 0.10
IMP_REENGAGE_GAIN = 0.00025
IMP_MAX_REENGAGE_BIAS = 0.0025
PEN_DOWN_ENGAGE_TIME = 0.25
PEN_DOWN_ENGAGE_BIAS = 0.0020
IMP_CONTACT_LOSS_SEARCH_BIAS = 0.0010
IMP_RECOVERY_RATE_BOOST = 2.0
CONTROL_MODE = "unknown_wall"
```

```
CONTACT_THRESHOLD = 0.5
CONTACT_LOST_THRESHOLD = 0.3
F_MAX_SAFE = 20.0
V_SEARCH = 0.003
V_RECOVERY = 0.004
K_FORCE = 0.0004
MAX_DX_PER_STEP = 0.001
MAX_APPROACH_DISTANCE = 0.06
MAX_RECOVERY_DISTANCE = 0.04
CONTACT_CONFIRM_STEPS = 5
CONTACT_LOST_CONFIRM_STEPS = 5
```

UNKNOWN_XCMD_LPF_ALPHA = IMP_XCMD_LPF_ALPHA

```
UNKNOWN_WALL_PRESETS = {
    "stable": {
        "contact_threshold": 0.5,
        "contact_lost_threshold": 0.3,
        "f_max_safe": 20.0,
        "v_search": 0.003,
        "v_recovery": 0.004,
        "k_force": 0.0004,
        "max_dx_per_step": 0.001,
        "max_approach_distance": 0.06,
        "max_recovery_distance": 0.04,
        "contact_confirm_steps": 5,
        "contact_lost_confirm_steps": 5,
        "unknown_xcmd_lpf_alpha": UNKNOWN_XCMD_LPF_ALPHA,
    },
    "aggressive": {
        "contact_threshold": 0.3,
        "contact_lost_threshold": 0.15,
        "f_max_safe": 20.0,
        "v_search": 0.010,
        "v_recovery": 0.012,
        "k_force": 0.0006,
        "max_dx_per_step": 0.0015,
        "max_approach_distance": 0.10,
        "max_recovery_distance": 0.08,
        "contact_confirm_steps": 2,
        "contact_lost_confirm_steps": 3,
        "unknown_xcmd_lpf_alpha": UNKNOWN_XCMD_LPF_ALPHA,
    },
}
UNKNOWN_WALL_PATH_TIME_BUDGET_SCALE = 4.0
UNKNOWN_MAX_XCMD_RATE = 0.22
```

Moving wall parameters:
The wall surface normal points along world +x, which is also the approach direction.
For a clean contact-control demo, the wall uses a one-sided motion profile:
it advances toward the brush, then retreats partway without going past its
initial x position, which helps keep the paint stroke continuous.

```
WALL_X_CENTER = 0.66
WALL_HALF_THICKNESS = 0.01
WALL_HALF_WIDTH_Y = 0.26
WALL_HALF_HEIGHT_Z = 0.22
WALL_PUSH_DISTANCE = 0.011
WALL_RETREAT_DISTANCE = 0.007
WALL_MOTION_START = 2.0
WALL_PUSH_DURATION = 2.0
WALL_RETURN_DURATION = 2.4
WALL_MOTION_ENABLED = True
WALL_PROFILE_MODE = "rigid"
CONCAVE_CENTER_Y = 0.0
CONCAVE_CENTER_Z = 0.30
CONCAVE_DEPTH = 0.0
CONCAVE_SIGMA = 0.05
BUMPY_AMP = 0.004
BUMPY_FREQ_Y = 22.0
BUMPY_FREQ_Z = 18.0
BUMPY_GRID_NY = 16
BUMPY_GRID_NZ = 12
```

Contact model parameters for the virtual spring-damper estimate.
CONTACT_STIFFNESS = 2500.0
CONTACT_DAMPING = 45.0
CONTACT_LOSS_FORCE_THRESHOLD = 0.4

Brush/tool geometry.
BRUSH_RADIUS = 0.015

```

# Simulation timing.
DT = 0.01
SIM_DURATION = 14.0

# Safety clamps for commanded normal motion.
MAX_DELTA_X = 0.05

RUN_PARAMETER_SWEEP = False

# Nominal painting path in the wall's tangential plane.
PAINT_Y_CENTER = 0.0
PAINT_Z_CENTER = 0.30
PAINT_Y_AMPLITUDE = 0.04
PAINT_Z_AMPLITUDE = 0.02
PAINT_FREQUENCY = 0.12

# UCLA letter painting parameters.
PAINT_TEXT = "UCLA"
LETTER_WIDTH = 0.085
LETTER_HEIGHT = 0.135
LETTER_SPACING = 0.030
LETTER_STROKE_SPEED = 0.085
PEN_UP_TRAVEL_SPEED = 0.12
PEN_UP_CLEARANCE = 0.012
PAINT_START_DELAY = 0.8
PAINT_END_HOLD = 1.0
PAINT_MARK_SPACING = 0.005
PAINT_MARK_RADIUS = 0.005
PAINT_MARK_COUNT = 900
PAINT_MARK_OFFSET = 0.0015
PAINT_MARK_HIDDEN_Y = 2.0
PAINT_MARK_RGBA = np.array([0.15, 0.34, 0.68, 1.0], dtype=float)
TEXT_READABLE_FROM_ROBOT_SIDE = True

# IK parameters.
IK_MAX_ITERS = 25
IK_DAMPING = 2.5e-3
IK_TOL = 1.0e-4
IK_STEP_LIMIT = 0.12

# Rendering parameters.
RENDER_VIDEO = True
RENDER_WIDTH = 480
RENDER_HEIGHT = 360
RENDER_FPS = 20
RENDER_EVERY_N_STEPS = max(1, int(round(1.0 / (RENDER_FPS * DT))))
SAVE_ROBOT_3D_ANIMATION = True
ROBOT_ANIMATION_FPS = 20
ROBOT_ANIMATION_EVERY_N_STEPS = max(1, int(round(1.0 / (ROBOT_ANIMATION_FPS * DT))))

# Native MuJoCo viewer parameters.
SHOW_NATIVE_MUJOCO_VIEWER = True
VIEWER_REALTIME = True
VIEWER_SHOW_LEFT_UI = False
VIEWER_SHOW_RIGHT_UI = False
KEEP_VIEWER_OPEN_AFTER_SIM = True
SHOW_CONTACT_VISUALS = False
SHOW_TOUCH_SITE_VISUAL = False
CAMERA_AZIMUTH = -38.0
CAMERA_ELEVATION = -11.0
CAMERA_DISTANCE = 0.96
CAMERA_LOOKAT = np.array([0.59, 0.0, PAINT_Z_CENTER + 0.02], dtype=float)
ROBOT_ANIMATION_ELEVATION = 18.0
ROBOT_ANIMATION_AZIMUTH = -40.0

# Saved render output.
SAVE_RENDER_GIF = True
SAVE_RENDER_MP4 = True

# Optional touch sensor usage. The manual contact estimate is usually more stable

```

```

# for this class-demo setup, so it is the default source.
USE_TOUCH_SENSOR = False

# If True, the demo approximates a very high-bandwidth inner joint servo by
# writing the IK solution directly into qpos before stepping MuJoCo. This keeps
# the notebook example stable and makes the impedance behavior easier to see.
USE_QUASI_STATIC_SERVO = True

# Paths and names for swapping in a real robot later.
EXTERNAL_MODEL_PATH = None
USE_REAL_PIPER_MODEL_IF_AVAILABLE = True
AUTO_FIND_LOCAL_PIPER_MODEL = True
AUTO_CLONE_PIPER_REPO = False
PIPER_REPO_URL = "https://github.com/agilexrobotics/piper_isaac_sim.git"
PIPER_REPO_BRANCH = "master"
PIPER_MODEL_RELATIVE_CANDIDATES = [
    Path("src/piper_description/mujoco_model/piper_description.xml"),
    Path("src/piper_description/mujoco_model/piper_no_gripper_description.xml"),
    Path("piper_description/mujoco_model/piper_description.xml"),
    Path("piper_description/mujoco_model/piper_no_gripper_description.xml"),
]
PIPER_ATTACHMENT_BODY_CANDIDATES = ["link6", "link7", "gripper_base", "tool_mount"]
EXTERNAL_BRUSH_OFFSET = np.array([0.0, 0.0, 0.165], dtype=float)
JOINT_NAMES = [f"joint{i}" for i in range(1, 7)]
EE_SITE_NAME = "brush_tip_site"
TOUCH_SENSOR_NAME = "brush_touch"
WALL_BODY_NAME = "moving_wall"

# Derived normal-direction references.
WALL_SURFACE_CENTER = WALL_X_CENTER - WALL_HALF_THICKNESS
DESIRED_PENETRATION = F_DES / CONTACT_STIFFNESS
X_NOMINAL_BASE = WALL_SURFACE_CENTER - BRUSH_RADIUS + DESIRED_PENETRATION
X_CMD_MIN = X_NOMINAL_BASE - 0.04
X_CMD_MAX = X_NOMINAL_BASE + 0.01

def paint_mark_sites_xml(indent=""):
    lines = []
    for index in range(PAINT_MARK_COUNT):
        lines.append(
            f'{indent}<site name="paint_mark_{index:04d}" '
            f'pos="{-WALL_HALF_THICKNESS - PAINT_MARK_OFFSET:.6f} {PAINT_MARK_HIDDEN_Y:.6f} 0" '
            f'size="{PAINT_MARK_RADIUS:.6f}" '
            f'rgba="0.15 0.34 0.68 0"/>'
        )
    return "\n".join(lines)

def wall_surface_offset_local(y_local, z_local):
    if WALL_PROFILE_MODE == "concave":
        dy = float(y_local - CONCAVE_CENTER_Y)
        dz = float(z_local - CONCAVE_CENTER_Z)
        r2 = dy * dy + dz * dz
        sigma2 = max(CONCAVE_SIGMA * CONCAVE_SIGMA, 1e-9)
        return float(-CONCAVE_DEPTH * math.exp(-r2 / sigma2))
    if WALL_PROFILE_MODE == "bumpy":
        return float(BUMPY_AMP * math.sin(BUMPY_FREQ_Y * y_local) * math.cos(BUMPY_FREQ_Z * (z_local - PAINT_Z_CENTER)))
    return 0.0

def wall_geoms_xml(indent=""):
    if WALL_PROFILE_MODE != "bumpy":
        return (
            f'{indent}<geom name="paint_wall" type="box" '
            f'size="{WALL_HALF_THICKNESS} {WALL_HALF_WIDTH_Y} {WALL_HALF_HEIGHT_Z}" '
            f'rgba="0.75 0.82 0.92 1" friction="0.8 0.02 0.002" contype="1" conaffinity="1"/>'
        )
    lines = []

```

```

dy = (2.0 * WALL_HALF_WIDTH_Y) / BUMPY_GRID_NY
dz = (2.0 * WALL_HALF_HEIGHT_Z) / BUMPY_GRID_NZ
tile_half_y = 0.5 * dy
tile_half_z = 0.5 * dz
for iy in range(BUMPY_GRID_NY):
    y_local = -WALL_HALF_WIDTH_Y + (iy + 0.5) * dy
    for iz in range(BUMPY_GRID_NZ):
        z_local = -WALL_HALF_HEIGHT_Z + (iz + 0.5) * dz
        z_world = PAINT_Z_CENTER + z_local
        x_offset = wall_surface_offset_local(y_local, z_world)
        thickness = max(0.002, WALL_HALF_THICKNESS - x_offset)
        x_center_local = x_offset
        lines.append(
            f'{indent}<geom name="paint_wall_{iy:02d}_{iz:02d}" type="box" '
            f'pos="{x_center_local:.6f} {y_local:.6f} {z_local:.6f}" '
            f'size="{thickness:.6f} {tile_half_y:.6f} {tile_half_z:.6f}" '
            f'rgba="0.75 0.82 0.92 1" friction="0.8 0.02 0.002" contype="1" conaffinity="1"/>'
        )
    return "\n".join(lines)

def local_to_world_yz(local_point_yz):
    world_y = PAINT_Y_CENTER - local_point_yz[0] if TEXT_READABLE_FROM_ROBOT_SIDE else PAINT_Y_CENTER
    + local_point_yz[0]
    return np.array(
        [X_NOMINAL_BASE, world_y, PAINT_Z_CENTER + local_point_yz[1]],
        dtype=float,
    )

def world_to_paint_plane_yz(world_y, world_z):
    local_y = PAINT_Y_CENTER - world_y if TEXT_READABLE_FROM_ROBOT_SIDE else world_y - PAINT_Y_CENTER
    local_z = world_z - PAINT_Z_CENTER
    return np.array([local_y, local_z], dtype=float)

def ucla_letter_strokes():
    """Return pen-down 2D polylines for drawing UCLA in the wall's y-z plane."""
    w = LETTER_WIDTH
    h = LETTER_HEIGHT
    s = LETTER_SPACING
    total_width = 4 * w + 3 * s
    x0 = -0.5 * total_width
    z_bottom = -0.5 * h
    z_top = 0.5 * h
    z_mid = 0.02 * h

    def letter_origin(letter_index):
        return x0 + letter_index * (w + s)

    strokes = []

    # U
    u0 = letter_origin(0)
    strokes.append(np.array([[u0, z_top], [u0, z_bottom], [u0 + w, z_bottom], [u0 + w, z_top]],
        dtype=float))

    # C
    c0 = letter_origin(1)
    strokes.append(
        np.array(
            [[c0 + w, z_top], [c0, z_top], [c0, z_bottom], [c0 + w, z_bottom]],
            dtype=float,
        )
    )

    # L
    l0 = letter_origin(2)
    strokes.append(np.array([[l0, z_top], [l0, z_bottom], [l0 + w, z_bottom]], dtype=float))

```

```

# A outline and crossbar.
a0 = letter_origin(3)
strokes.append(np.array([[a0, z_bottom], [a0 + 0.5 * w, z_top], [a0 + w, z_bottom]], dtype=float))
strokes.append(np.array([[a0 + 0.25 * w, z_mid], [a0 + 0.75 * w, z_mid]], dtype=float))

return strokes

def polyline_length(points):
    if len(points) < 2:
        return 0.0
    return float(np.sum(np.linalg.norm(np.diff(points, axis=0), axis=1)))

def interpolate_polyline(points, distance_along):
    if len(points) == 1:
        return points[0].copy()

    remaining = float(np.clip(distance_along, 0.0, polyline_length(points)))
    for start, end in zip(points[:-1], points[1:]):
        segment = end - start
        segment_length = float(np.linalg.norm(segment))
        if segment_length < 1e-9:
            continue
        if remaining <= segment_length:
            return start + (remaining / segment_length) * segment
        remaining -= segment_length
    return points[-1].copy()

def build_paint_plan():
    """Build a timed path that paints UCLA with pen-up travel between strokes."""
    strokes = ucla_letter_strokes()
    segments = []
    cursor = None

    for stroke in strokes:
        if cursor is not None and np.linalg.norm(stroke[0] - cursor) > 1e-9:
            segments.append({"points": np.vstack([cursor, stroke[0]]), "pen_down": False, "speed":
PEN_UP_TRAVEL_SPEED})
        segments.append({"points": stroke, "pen_down": True, "speed": LETTER_STROKE_SPEED})
        cursor = stroke[-1].copy()

    cumulative_time = PAINT_START_DELAY
    timed_segments = []
    for segment in segments:
        length = polyline_length(segment["points"])
        duration = 0.0 if length < 1e-9 else length / segment["speed"]
        timed_segments.append(
            {
                "points": segment["points"],
                "pen_down": segment["pen_down"],
                "speed": segment["speed"],
                "length": length,
                "t_start": cumulative_time,
                "t_end": cumulative_time + duration,
            }
        )
        cumulative_time += duration

    total_duration = cumulative_time + PAINT_END_HOLD
    return {
        "segments": timed_segments,
        "total_duration": total_duration,
        "start_point": local_to_world_yz(strokes[0][0]),
        "end_point": local_to_world_yz(strokes[-1][-1]),
    }

PAINT_PLAN = build_paint_plan()

```

```

# %% Model construction
def build_simplified_piper_like_mjcf() -> str:
    """Return a self-contained MJCF string for a simple 6-DOF serial arm."""
    return dedent(
        f"""
        <mujoco model="piper_like_painting_arm">
          <compiler angle="degree" autolimits="true"/>
          <option timestep="{DT}" gravity="0 0 0" integrator="RK4" iterations="80"
ls_iterations="20"/>
          <size njmax="1000" nconmax="500"/>

          <visual>
            <headlight ambient="0.45 0.45 0.45" diffuse="0.85 0.85 0.85" specular="0.15 0.15 0.15"/>
            <rgba haze="0.98 0.99 1.0 1"/>
          </visual>

          <asset>
            <texture type="skybox" builtin="gradient" rgb1="0.92 0.96 1.0" rgb2="0.58 0.70 0.90"
width="256" height="256"/>
          </asset>

          <default>
            <joint damping="3.0" armature="0.02"/>
            <geom density="550"/>
          </default>

          <worldbody>
            <light pos="0.0 -1.0 1.6" dir="0 1 -1"/>
            <geom name="ground" type="plane" pos="0 0 0" size="2 2 0.1"
              rgba="0.96 0.96 0.96 1" contype="0" conaffinity="0"/>

            <body name="{WALL_BODY_NAME}" mocap="true" pos="{WALL_X_CENTER} 0 {PAINT_Z_CENTER}">
{wall_geoms_xml()}
{paint_mark_sites_xml()}
            </body>

            <body name="robot_base" pos="0 0 {PAINT_Z_CENTER}">
              <geom name="base_geom" type="cylinder" size="0.055 0.05"
                rgba="0.16 0.16 0.20 1" contype="0" conaffinity="0"/>

              <body name="link1" pos="0 0 0">
                <joint name="joint1" axis="0 0 1" range="-170 170"/>
                <geom name="link1_geom" type="capsule" fromto="0 0 0 0.14 0 0"
                  size="0.022" rgba="0.29 0.43 0.73 1" contype="0" conaffinity="0"/>

                <body name="link2" pos="0.14 0 0">
                  <joint name="joint2" axis="0 1 0" range="-110 110"/>
                  <geom name="link2_geom" type="capsule" fromto="0 0 0 0.12 0 0"
                    size="0.020" rgba="0.33 0.47 0.78 1" contype="0" conaffinity="0"/>

                  <body name="link3" pos="0.12 0 0">
                    <joint name="joint3" axis="0 1 0" range="-130 130"/>
                    <geom name="link3_geom" type="capsule" fromto="0 0 0 0.12 0 0"
                      size="0.019" rgba="0.37 0.51 0.82 1" contype="0" conaffinity="0"/>

                    <body name="link4" pos="0.12 0 0">
                      <joint name="joint4" axis="1 0 0" range="-180 180"/>
                      <geom name="link4_geom" type="capsule" fromto="0 0 0 0.10 0 0"
                        size="0.017" rgba="0.41 0.55 0.86 1" contype="0" conaffinity="0"/>

                      <body name="link5" pos="0.10 0 0">
                        <joint name="joint5" axis="0 1 0" range="-130 130"/>
                        <geom name="link5_geom" type="capsule" fromto="0 0 0 0.08 0 0"
                          size="0.015" rgba="0.46 0.60 0.90 1" contype="0" conaffinity="0"/>

                        <body name="link6" pos="0.08 0 0">
                          <joint name="joint6" axis="1 0 0" range="-180 180"/>
                          <geom name="link6_geom" type="capsule" fromto="0 0 0 0.06 0 0"

```



```
size="0.014" rgba="0.50 0.64 0.94 1" contype="0" conaffinity="0"/>
```

```
<body name="tool_mount" pos="0.06 0 0">
  <geom name="wrist_geom" type="sphere" size="0.022"
    rgba="0.30 0.30 0.34 1" contype="0" conaffinity="0"/>
  <geom name="brush_tip_geom" pos="0.02 0 0" type="sphere" size="
    rgba="0.95 0.28 0.20 1" friction="1.0 0.02 0.002"
    contype="1" conaffinity="1"/>
  <site name="{EE_SITE_NAME}" pos="0.02 0 0" size="0.010"
    rgba="1.0 0.0 0.0 1"/>
  <site name="brush_touch_site" pos="0.02 0 0" size="0.025"
    rgba="0.2 0.5 1.0 {0.20 if SHOW_TOUCH_SITE_VISUAL else 0.0}"/>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</body>
```

```
</worldbody>
```

```
<actuator>
```

```
  <position name="act1" joint="joint1" kp="180"/>
```

```
  <position name="act2" joint="joint2" kp="180"/>
```

```
  <position name="act3" joint="joint3" kp="160"/>
```

```
  <position name="act4" joint="joint4" kp="110"/>
```

```
  <position name="act5" joint="joint5" kp="100"/>
```

```
  <position name="act6" joint="joint6" kp="90"/>
```

```
</actuator>
```

```
<sensor>
```

```
  <touch name="{TOUCH_SENSOR_NAME}" site="brush_touch_site"/>
```

```
</sensor>
```

```
</mujoco>
```

```
"""
```

```
)
```

```
def maybe_clone_piper_repo(repo_root: Path):
```

```
    """Optionally clone the Piper ROS repository when running in Colab."""
```

```
    if repo_root.exists() or not AUTO_CLONE_PIPER_REPO:
```

```
        return repo_root if repo_root.exists() else None
```

```
    try:
```

```
        subprocess.check_call([
```

```
            [
```

```
                "git",
```

```
                "clone",
```

```
                "--depth",
```

```
                "1",
```

```
                "--branch",
```

```
                PIPER_REPO_BRANCH,
```

```
                PIPER_REPO_URL,
```

```
                str(repo_root),
```

```
            ]
```

```
        ]
```

```
        return repo_root
```

```
    except Exception as exc:
```

```
        print(f"Could not clone Piper repository automatically. Falling back if needed. Reason:
```

```
{exc}")
```

```
        return None
```

```
def candidate_piper_roots():
```

```
    """Common places where the Piper ROS repository may exist locally."""
```

```
    roots = []
```

```
    for env_name in ("PIPER_ASSET_REPO_DIR", "PIPER_ROS_DIR"):
```

```
        env_root = os.getenv(env_name)
```

```

if env_root:
    roots.append(Path(env_root))

cwd = Path.cwd()
roots.extend(
    [
        SCRIPT_DIR,
        SCRIPT_DIR / "piper_isaac_sim",
        SCRIPT_DIR / "piper_ros",
        SCRIPT_DIR.parent / "piper_isaac_sim",
        SCRIPT_DIR.parent / "piper_ros",
        cwd,
        cwd / "piper_isaac_sim",
        cwd / "piper_ros",
        cwd.parent / "piper_isaac_sim",
        cwd.parent / "piper_ros",
        Path("/content/piper_isaac_sim"),
        Path("/content/piper_ros"),
        Path.home() / "piper_isaac_sim",
        Path.home() / "piper_ros",
        Path.home() / "Desktop" / "piper_isaac_sim",
        Path.home() / "Desktop" / "piper_ros",
    ]
)

```

```

unique_roots = []
seen = set()
for root in roots:
    if root is None:
        continue
    resolved = root.expanduser()
    key = str(resolved)
    if key not in seen:
        seen.add(key)
        unique_roots.append(resolved)
return unique_roots

```

```

def find_local_piper_model():
    """Return the first discovered local Piper MuJoCo XML file."""
    for root in candidate_piper_roots():
        should_try_clone = str(root).endswith("/content/piper_isaac_sim") or
str(root).endswith("/content/piper_ros")
        actual_root = maybe_clone_piper_repo(root) if should_try_clone else root
        if actual_root is None:
            continue
        for relative_path in PIPER_MODEL_RELATIVE_CANDIDATES:
            candidate = actual_root / relative_path
            if candidate.exists():
                return candidate
    return None

```

```

def format_xyz(values):
    return " ".join(f"{float(v):.6f}" for v in values)

```

```

def absolutize_asset_paths(xml_root: ET.Element, source_dir: Path):
    for tag_name in ("mesh", "texture"):
        for element in xml_root.findall(f".//{tag_name}"):
            file_attr = element.attrib.get("file")
            if not file_attr:
                continue
            element.set("file", str((source_dir / file_attr).resolve()))

```

```

def ensure_xml_section(xml_root: ET.Element, section_name: str):
    section = xml_root.find(section_name)
    if section is None:
        section = ET.SubElement(xml_root, section_name)

```

```
return section
```

```
def find_ee_attachment_body(xml_root: ET.Element):
    for body_name in PIPER_ATTACHMENT_BODY_CANDIDATES:
        body = xml_root.find(f".//body[@name='{body_name}']")
        if body is not None:
            return body

    bodies = xml_root.findall(".//body")
    if not bodies:
        raise ValueError("No robot bodies were found in the external Piper XML.")
    return bodies[-1]
```

```
def ensure_position_actuators(xml_root: ET.Element):
    actuator = ensure_xml_section(xml_root, "actuator")
    existing_joint_actuators = {node.attrib.get("joint") for node in actuator}
    for joint_name in JOINT_NAMES:
        if joint_name in existing_joint_actuators:
            continue
        ET.SubElement(
            actuator,
            "position",
            attrib={
                "name": f"{joint_name}_paint_ctrl",
                "joint": joint_name,
                "kp": "2000",
            },
        )
```

```
def patch_external_piper_model(external_model_path: Path):
    """
```

Patch the vendor Piper MuJoCo XML into a self-contained painting scene.

The vendor file provides the robot and actuators. This helper adds:

- the moving wall,
 - a brush-tip body/geom/site,
 - a touch sensor, and
 - absolute mesh paths so the patched XML can be written anywhere.
- ```
"""
```

```
tree = ET.parse(external_model_path)
root = tree.getroot()
source_dir = external_model_path.parent
```

```
absolutize_asset_paths(root, source_dir)
```

```
option = ensure_xml_section(root, "option")
option.set("timestep", str(DT))
```

```
worldbody = ensure_xml_section(root, "worldbody")
```

```
if root.find(".//geom[@name='ground']") is None:
```

```
 ET.SubElement(
 worldbody,
 "geom",
 attrib={
 "name": "ground",
 "type": "plane",
 "pos": "0 0 0",
 "size": "2 2 0.1",
 "rgba": "0.96 0.96 0.96 1",
 "contype": "0",
 "conaffinity": "0",
 },
)
```

```
wall_body = root.find(f".//body[@name='{WALL_BODY_NAME}']")
if wall_body is None:
```

```

wall_body = ET.SubElement(
 worldbody,
 "body",
 attrib={
 "name": WALL_BODY_NAME,
 "mocap": "true",
 "pos": format_xyz([WALL_X_CENTER, 0.0, PAINT_Z_CENTER]),
 },
)
ET.SubElement(
 wall_body,
 "geom",
 attrib={
 "name": "paint_wall",
 "type": "box",
 "size": format_xyz([WALL_HALF_THICKNESS, WALL_HALF_WIDTH_Y, WALL_HALF_HEIGHT_Z]),
 "rgba": "0.75 0.82 0.92 1",
 "friction": "0.8 0.02 0.002",
 "contype": "1",
 "conaffinity": "1",
 },
)

for index in range(PAINT_MARK_COUNT):
 site_name = f"paint_mark_{index:04d}"
 if wall_body.find(f"//site[@name='{site_name}']") is None:
 ET.SubElement(
 wall_body,
 "site",
 attrib={
 "name": site_name,
 "pos": format_xyz([- (WALL_HALF_THICKNESS + PAINT_MARK_OFFSET),
PAINT_MARK_HIDDEN_Y, 0.0]),
 "size": f"{PAINT_MARK_RADIUS:.6f}",
 "rgba": "0.15 0.34 0.68 0",
 },
)

attachment_body = find_ee_attachment_body(root)
if attachment_body.find(f"//site[@name='{EE_SITE_NAME}']") is None:
 tool_mount = ET.SubElement(
 attachment_body,
 "body",
 attrib={
 "name": "painting_tool_mount",
 "pos": format_xyz(EXTERNAL_BRUSH_OFFSET),
 },
)
 ET.SubElement(
 tool_mount,
 "geom",
 attrib={
 "name": "brush_tip_geom",
 "type": "sphere",
 "size": f"{BRUSH_RADIUS:.6f}",
 "rgba": "0.95 0.28 0.20 1",
 "friction": "1.0 0.02 0.002",
 "contype": "1",
 "conaffinity": "1",
 },
)
 ET.SubElement(
 tool_mount,
 "site",
 attrib={
 "name": EE_SITE_NAME,
 "size": "0.010",
 "rgba": "1.0 0.0 0.0 1",
 },
)

```

```

 ET.SubElement(
 tool_mount,
 "site",
 attrib={
 "name": "brush_touch_site",
 "size": "0.025",
 "rgba": f"0.2 0.5 1.0 {0.20 if SHOW_TOUCH_SITE_VISUAL else 0.0}",
 },
)

 sensor = ensure_xml_section(root, "sensor")
 existing_sensor_names = {node.attrib.get("name") for node in sensor}
 if TOUCH_SENSOR_NAME not in existing_sensor_names:
 ET.SubElement(
 sensor,
 "touch",
 attrib={
 "name": TOUCH_SENSOR_NAME,
 "site": "brush_touch_site",
 },
)

 ensure_position_actuators(root)

 patched_dir = Path(tempfile.gettempdir())
 patched_path = patched_dir / "patched_piper_painting_scene.xml"
 tree.write(patched_path, encoding="utf-8", xml_declaration=True)
 return patched_path

def resolve_external_model_path():
 if EXTERNAL_MODEL_PATH is not None:
 candidate = Path(EXTERNAL_MODEL_PATH).expanduser()
 if candidate.is_dir():
 for relative_path in PIPER_MODEL_RELATIVE_CANDIDATES:
 nested = candidate / relative_path
 if nested.exists():
 return nested, f"user-specified Piper repo: {nested}"
 elif candidate.exists():
 return candidate, f"user-specified model: {candidate}"

 if USE_REAL_PIPER_MODEL_IF_AVAILABLE and AUTO_FIND_LOCAL_PIPER_MODEL:
 candidate = find_local_piper_model()
 if candidate is not None:
 return candidate, f"auto-discovered Piper MuJoCo model: {candidate}"

 return None, None

def load_model():
 """Load an external model if provided, otherwise build the self-contained demo."""
 external_model_path, external_source = resolve_external_model_path()
 if external_model_path is not None:
 try:
 patched_model_path = patch_external_piper_model(external_model_path)
 model = mujoco.MjModel.from_xml_path(str(patched_model_path))
 source = f"{external_source} patched with wall and brush"
 return model, source
 except Exception as exc:
 print(f"Could not use the external Piper model at {external_model_path}. Falling back. Reason: {exc}")

 model = mujoco.MjModel.from_xml_string(build_simplified_piper_like_mjcf())
 source = "self-contained simplified 6-DOF Piper-like MJCF"
 return model, source

%% Kinematics helpers
def get_joint_indices(model, joint_names):
 qpos_indices = []

```

```

dof_indices = []
lower = []
upper = []
for name in joint_names:
 joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, name)
 if joint_id < 0:
 raise ValueError(
 f"Joint '{name}' was not found. Update JOINT_NAMES when swapping in a real robot
model."
)
 qpos_indices.append(model.jnt_qposadr[joint_id])
 dof_indices.append(model.jnt_dofadr[joint_id])
 lower.append(model.jnt_range[joint_id, 0])
 upper.append(model.jnt_range[joint_id, 1])
return (
 np.array(qpos_indices, dtype=int),
 np.array(dof_indices, dtype=int),
 np.array(lower, dtype=float),
 np.array(upper, dtype=float),
)

def get_joint_actuator_indices(model, joint_names):
 """Map each commanded joint to its corresponding actuator index."""
 actuator_indices = []
 for joint_name in joint_names:
 joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, joint_name)
 if joint_id < 0:
 raise ValueError(f"Joint '{joint_name}' was not found while searching for actuators.")

 matching_actuator_id = None
 for actuator_id in range(model.nu):
 transmission_target = int(model.actuator_trnid[actuator_id, 0])
 if transmission_target == joint_id:
 matching_actuator_id = actuator_id
 break

 if matching_actuator_id is None:
 raise ValueError(
 f"No actuator was found for joint '{joint_name}'. "
 "Check the external model actuators or update JOINT_NAMES."
)
 actuator_indices.append(matching_actuator_id)

 return np.array(actuator_indices, dtype=int)

def get_ee_pose(model, data, site_id):
 """Return the current end-effector Cartesian position."""
 return data.site_xpos[site_id].copy()

def get_robot_chain_body_ids(model, joint_names):
 """Return body ids for the serial chain associated with the commanded joints."""
 body_ids = []
 for name in joint_names:
 joint_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, name)
 if joint_id < 0:
 continue
 body_id = model.jnt_bodyid[joint_id]
 if body_id not in body_ids:
 body_ids.append(body_id)
 return body_ids

def get_robot_chain_points(model, data, body_ids, site_id):
 """Return a list of 3D points that sketch the robot chain for animation."""
 points = []
 for body_id in body_ids:
 points.append(data.xpos[body_id].copy())

```

```

points.append(data.site_xpos[site_id].copy())
return np.asarray(points, dtype=float)

```

```

def compute_jacobian(model, data, site_id):
 """Return translational and rotational Jacobians for the end-effector site."""
 jaccp = np.zeros((3, model.nv))
 jacr = np.zeros((3, model.nv))
 mujoco.mj_jacSite(model, data, jaccp, jacr, site_id)
 return jaccp, jacr

```

```

def get_site_linear_velocity(model, data, site_id):
 """Estimate site linear velocity from the translational Jacobian."""
 jaccp, _ = compute_jacobian(model, data, site_id)
 return jaccp @ data.qvel

```

```

def ik_damped_least_squares(
 model,
 ik_data,
 site_id,
 target_pos,
 q_seed,
 qpos_indices,
 dof_indices,
 joint_lower,
 joint_upper,
 damping=IK_DAMPING,
 max_iters=IK_MAX_ITERS,
 tol=IK_TOL,
):
 """
 Solve position-only inverse kinematics with a damped least-squares update.

 This keeps the code simple and easy to replace if you later use a different
 IK package or a real Piper model with task-space control.
 """
 q_trial = q_seed.copy()

 for _ in range(max_iters):
 ik_data.qpos[qpos_indices] = q_trial
 ik_data.qvel[:] = 0.0
 mujoco.mj_forward(model, ik_data)

 current_pos = ik_data.site_xpos[site_id].copy()
 position_error = target_pos - current_pos
 if np.linalg.norm(position_error) < tol:
 break

 jaccp = np.zeros((3, model.nv))
 jacr = np.zeros((3, model.nv))
 mujoco.mj_jacSite(model, ik_data, jaccp, jacr, site_id)
 J = jaccp[:, dof_indices]

 regularized = J @ J.T + (damping**2) * np.eye(3)
 dq = J.T @ np.linalg.solve(regularized, position_error)
 dq = np.clip(dq, -IK_STEP_LIMIT, IK_STEP_LIMIT)

 q_trial = np.clip(q_trial + dq, joint_lower, joint_upper)

 return q_trial

```

```

%% Motion and control helpers
def smoothstep_fraction(t, t_start, duration):
 """Return a smooth 0->1 phase and its time derivative over the interval."""
 if duration <= 1e-9:
 return (1.0, 0.0) if t >= t_start else (0.0, 0.0)

```

```

if t <= t_start:
 return 0.0, 0.0
if t >= t_start + duration:
 return 1.0, 0.0

s = (t - t_start) / duration
phase = s * s * (3.0 - 2.0 * s)
phase_dot = (6.0 * s * (1.0 - s)) / duration
return float(phase), float(phase_dot)

```

```

def wall_motion(t):
 """
 Showcase-friendly wall motion along x.

 The wall starts at its nominal x location, moves toward the robot to
 increase contact force, then retreats partway while staying no farther away
 than its original x position. This makes the impedance response easy to see
 while keeping the painting task feasible.
 """
 if not WALL_MOTION_ENABLED:
 return float(WALL_X_CENTER), 0.0

 push_phase, push_phase_dot = smoothstep_fraction(t, WALL_MOTION_START, WALL_PUSH_DURATION)
 return_phase, return_phase_dot = smoothstep_fraction(
 t,
 WALL_MOTION_START + WALL_PUSH_DURATION,
 WALL_RETURN_DURATION,
)

 x = WALL_X_CENTER - WALL_PUSH_DISTANCE * push_phase + WALL_RETREAT_DISTANCE * return_phase
 vx = -WALL_PUSH_DISTANCE * push_phase_dot + WALL_RETREAT_DISTANCE * return_phase_dot
 return float(x), float(vx)

```

```

def wall_surface_offset_for_yz(y, z):
 return wall_surface_offset_local(float(y), float(z))

```

```

def effective_sim_duration():
 return max(SIM_DURATION, PAINT_PLAN["total_duration"])

```

```

def painting_nominal_pose(t):
 """
 Return the nominal position and pen-down state for painting UCLA.

 When the brush is between strokes, pen_down=False so the controller can back
 away from the wall slightly instead of drawing connecting travel lines.
 """
 if t <= PAINT_START_DELAY:
 return PAINT_PLAN["start_point"].copy(), False

 for segment in PAINT_PLAN["segments"]:
 if segment["t_start"] <= t <= segment["t_end"] or (
 abs(t - segment["t_end"]) < 1e-9 and segment["t_end"] >= segment["t_start"]
):
 distance = segment["speed"] * max(0.0, t - segment["t_start"])
 local_point_yz = interpolate_polyline(segment["points"], distance)
 return local_to_world_yz(local_point_yz), segment["pen_down"]

 return PAINT_PLAN["end_point"].copy(), False

```

```

def get_paint_mark_site_ids(model):
 site_ids = []
 for index in range(PAINT_MARK_COUNT):
 site_name = f"paint_mark_{index:04d}"
 site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, site_name)
 if site_id >= 0:

```



```
 site_ids.append(site_id)
 return site_ids
```

```
def reset_paint_marks(model, site_ids):
 hidden_pos = np.array([(WALL_HALF_THICKNESS + PAINT_MARK_OFFSET), PAINT_MARK_HIDDEN_Y, 0.0],
dtype=float)
 for site_id in site_ids:
 model.site_pos[site_id] = hidden_pos
 model.site_rgba[site_id] = np.array([PAINT_MARK_RGBA[0], PAINT_MARK_RGBA[1],
PAINT_MARK_RGBA[2], 0.0])
```

```
def maybe_add_paint_mark(model, site_ids, next_mark_index, ee_pos, last_mark_local_yz):
 if next_mark_index >= len(site_ids):
 return next_mark_index, last_mark_local_yz

 local_yz = world_to_paint_plane_yz(ee_pos[1], ee_pos[2])
 if abs(local_yz[0]) > WALL_HALF_WIDTH_Y or abs(local_yz[1]) > WALL_HALF_HEIGHT_Z:
 return next_mark_index, last_mark_local_yz
 if last_mark_local_yz is not None and np.linalg.norm(local_yz - last_mark_local_yz) <
PAINT_MARK_SPACING:
 return next_mark_index, last_mark_local_yz

 site_id = site_ids[next_mark_index]
 site_y = PAINT_Y_CENTER - local_yz[0] if TEXT_READABLE_FROM_ROBOT_SIDE else PAINT_Y_CENTER +
local_yz[0]
 model.site_pos[site_id] = np.array([(WALL_HALF_THICKNESS + PAINT_MARK_OFFSET), site_y,
local_yz[1]], dtype=float)
 model.site_rgba[site_id] = PAINT_MARK_RGBA
 return next_mark_index + 1, local_yz
```

```
def get_touch_sensor_value(model, data, sensor_name):
 sensor_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SENSOR, sensor_name)
 if sensor_id < 0:
 return None
 adr = model.sensor_adr[sensor_id]
 dim = model.sensor_dim[sensor_id]
 values = data.sensordata[adr : adr + dim]
 if dim == 1:
 return float(values[0])
 return values.copy()
```

```
def contact_force(model, data, site_id, t):
 """
 Estimate the normal contact force.

 Preferred option in many MuJoCo demos is a touch sensor, but in this
 assignment the manual spring-damper estimate is often more stable and easier
 to interpret. The helper keeps both paths available.
 """
 ee_pos = get_ee_pose(model, data, site_id)
 ee_vel = get_site_linear_velocity(model, data, site_id)

 wall_x, wall_vx = wall_motion(t)
 wall_surface_x = wall_x - WALL_HALF_THICKNESS + wall_surface_offset_for_yz(ee_pos[1], ee_pos[2])

 penetration = ee_pos[0] + BRUSH_RADIUS - wall_surface_x
 closing_speed = ee_vel[0] - wall_vx

 if penetration > 0.0:
 force_manual = CONTACT_STIFFNESS * penetration + CONTACT_DAMPING * closing_speed
 force_manual = max(0.0, force_manual)
 else:
 force_manual = 0.0

 force_touch = get_touch_sensor_value(model, data, TOUCH_SENSOR_NAME)
 if force_touch is None:
```

```
force_touch = 0.0
```

```
force_used = float(force_touch) if USE_TOUCH_SENSOR else float(force_manual)
```

```
return {
 "force_used": force_used,
 "force_manual": float(force_manual),
 "force_touch": float(force_touch),
 "penetration": float(max(0.0, penetration)),
 "wall_x": float(wall_x),
 "wall_surface_x": float(wall_surface_x),
 "wall_vx": float(wall_vx),
 "ee_x": float(ee_pos[0]),
}
```

```
%% Plotting and rendering
```

```
def plot_results(log, output_dir):
 time = np.asarray(log["time"])
 wall_surface_x = np.asarray(log["wall_surface_x"])
 ee_x = np.asarray(log["ee_x"])
 x_cmd = np.asarray(log["x_cmd"])
 force_meas = np.asarray(log["force_meas"])
 force_des = np.asarray(log["force_des"])
 contact_lost = np.asarray(log["contact_lost"], dtype=float)
 ee_y = np.asarray(log["ee_y"])
 ee_z = np.asarray(log["ee_z"])
 pen_down = np.asarray(log["pen_down"], dtype=bool)
 joint_positions = np.asarray(log["joint_positions"])
 joint_commands = np.asarray(log["joint_commands"])

 true_wall_surface_x = np.asarray(log["true_wall_surface_x"])
 force_error = np.asarray(log["force_error"])
 controller_state = np.asarray(log["controller_state"])
 state_to_int = {"APPROACH": 0, "PAINTING": 1, "RECOVERY": 2, "SAFETY_STOP": 3}
 state_values = np.array([state_to_int.get(s, -1) for s in controller_state], dtype=float)

 fig1, axes = plt.subplots(5, 1, figsize=(10, 14), sharex=True)
 axes[0].plot(time, state_values, linewidth=1.8, color="tab:blue")
 axes[0].set_yticks([0, 1, 2, 3], ["APPROACH", "PAINTING", "RECOVERY", "SAFETY_STOP"])
 axes[0].set_ylabel("state")
 axes[0].set_title("Unknown-wall controller state")
 axes[0].grid(True, alpha=0.3)

 axes[1].plot(time, force_meas, label="Measured contact force", linewidth=2)
 axes[1].plot(time, force_des, label="Desired contact force", linestyle="--")
 axes[1].set_ylabel("Force [N]")
 axes[1].set_title("Force tracking")
 axes[1].grid(True, alpha=0.3)
 axes[1].legend()

 axes[2].plot(time, x_cmd, label="x_cmd", linewidth=2)
 axes[2].plot(time, ee_x, label="ee_x", linewidth=2)
 axes[2].plot(time, true_wall_surface_x, label="true_wall_surface_x (eval only)", linestyle="--")
 axes[2].set_ylabel("x [m]")
 axes[2].set_title("Command vs EE x vs true wall surface")
 axes[2].grid(True, alpha=0.3)
 axes[2].legend()

 axes[3].plot(time, contact_lost, color="tab:red", linewidth=1.8, label="contact_lost")
 axes[3].set_ylabel("flag")
 axes[3].set_ylim(-0.1, 1.1)
 axes[3].set_title("Contact-lost flag")
 axes[3].grid(True, alpha=0.3)
 axes[3].legend()

 axes[4].plot(time, force_error, color="tab:purple", linewidth=1.8, label="force_error")
 axes[4].set_xlabel("Time [s]")
 axes[4].set_ylabel("N")
 axes[4].set_title("Force error (F_DES - F_measured)")
```

```

axes[4].grid(True, alpha=0.3)
axes[4].legend()

fig1.tight_layout()
fig1_path = output_dir / "impedance_timeseries.png"
fig1.savefig(fig1_path, dpi=180, bbox_inches="tight")

fig2, axes2 = plt.subplots(1, 2, figsize=(13, 5))
painted_coords = np.array([world_to_paint_plane_yz(y, z) for y, z in zip(ee_y, ee_z)])
painted_y = painted_coords[:, 0]
painted_z = painted_coords[:, 1]
painted_y[~pen_down] = np.nan
painted_z[~pen_down] = np.nan
axes2[0].plot(painted_y, painted_z, label=f'Painted path: "{PAINT_TEXT}"', linewidth=2.5)
axes2[0].set_xlabel("wall-horizontal coordinate [m]")
axes2[0].set_ylabel("wall-vertical coordinate [m]")
axes2[0].set_title(f'Painting path in the wall plane: "{PAINT_TEXT}"')
axes2[0].axis("equal")
axes2[0].grid(True, alpha=0.3)
axes2[0].legend()

for i in range(joint_positions.shape[1]):
 axes2[1].plot(time, joint_positions[:, i], label=f"q{i+1}")
 axes2[1].plot(time, joint_commands[:, i], linestyle="--", alpha=0.55)
axes2[1].set_xlabel("Time [s]")
axes2[1].set_ylabel("Joint angle [rad]")
axes2[1].set_title("Joint trajectories (solid = actual, dashed = command)")
axes2[1].grid(True, alpha=0.3)
axes2[1].legend(ncol=2, fontsize=8)

fig2.tight_layout()
fig2_path = output_dir / "painting_path_and_joints.png"
fig2.savefig(fig2_path, dpi=180, bbox_inches="tight")

fig3 = plt.figure(figsize=(8, 6))
ax3 = fig3.add_subplot(111, projection="3d")
ax3.plot(np.asarray(log["ee_x"]), ee_y, ee_z, linewidth=2, label="Brush tip")
ax3.set_xlabel("x [m]")
ax3.set_ylabel("y [m]")
ax3.set_zlabel("z [m]")
ax3.set_title("3D end-effector trajectory")
ax3.legend()
fig3.tight_layout()
fig3_path = output_dir / "ee_3d_path.png"
fig3.savefig(fig3_path, dpi=180, bbox_inches="tight")

if "agg" not in plt.get_backend().lower():
 plt.show()
else:
 plt.close("all")
return fig1_path, fig2_path, fig3_path

```

```

def make_scene_option():
 option = mujoco.MjvOption()
 set_vis_flag(option, "mjVIS_SITE", True)
 set_vis_flag(option, "mjVIS_CONTACTPOINT", SHOW_CONTACT_VISUALS)
 set_vis_flag(option, "mjVIS_CONTACTFORCE", SHOW_CONTACT_VISUALS)
 return option

```

```

def make_renderer(model):
 renderer = mujoco.Renderer(model, height=RENDER_HEIGHT, width=RENDER_WIDTH)
 camera = mujoco.MjvCamera()
 camera.type = mujoco.mjtCamera.mjCAMERA_FREE
 camera.azimuth = CAMERA_AZIMUTH
 camera.elevation = CAMERA_ELEVATION
 camera.distance = CAMERA_DISTANCE
 camera.lookat[:] = CAMERA_LOOKAT
 return renderer, camera, make_scene_option()

```

```

def running_with_mjpython():
 return bool(os.getenv("MJPYTHON_BIN")) or "mjpython" in Path(sys.argv[0]).name.lower()

def should_launch_native_viewer():
 return SHOW_NATIVE_MUJOOCO_VIEWER and not IN_COLAB

def configure_native_viewer_camera(viewer_handle):
 with viewer_handle.lock():
 viewer_handle.cam.azimuth = CAMERA_AZIMUTH
 viewer_handle.cam.elevation = CAMERA_ELEVATION
 viewer_handle.cam.distance = CAMERA_DISTANCE
 viewer_handle.cam.lookat[:] = CAMERA_LOOKAT
 set_vis_flag(viewer_handle.opt, "mjVIS_SITE", True)
 set_vis_flag(viewer_handle.opt, "mjVIS_CONTACTPOINT", SHOW_CONTACT_VISUALS)
 set_vis_flag(viewer_handle.opt, "mjVIS_CONTACTFORCE", SHOW_CONTACT_VISUALS)

def maybe_launch_native_viewer(model, data):
 """
 Launch the MuJoCo mesh viewer when possible.

 Official MuJoCo docs note that on macOS `launch_passive` must be executed via
 the special `mjpython` launcher rather than plain `python`.
 """
 if not should_launch_native_viewer():
 return None

 if sys.platform == "darwin" and not running_with_mjpython():
 print(
 "Native MuJoCo viewer not launched because macOS passive viewer requires `mjpython`.\\n"
 "Run this script with:\\n"
 f" /opt/miniconda3/bin/mjpython {SCRIPT_DIR / 'MuJoCo_test.py'}"
)
 return None

 viewer_handle = mujoco_viewer.launch_passive(
 model,
 data,
 show_left_ui=VIEWER_SHOW_LEFT_UI,
 show_right_ui=VIEWER_SHOW_RIGHT_UI,
)
 configure_native_viewer_camera(viewer_handle)
 return viewer_handle

def should_attempt_rendering():
 """Attempt offscreen MuJoCo rendering whenever video export is enabled."""
 return RENDER_VIDEO

def save_render_outputs(frames, output_dir):
 if not frames:
 return None, None

 mp4_path = None
 gif_path = None

 if SAVE_RENDER_MP4:
 candidate_path = output_dir / "painting_demo.mp4"
 if imageio_ffmpeg is None:
 print("imageio-ffmpeg is not available, so MP4 export is skipped and GIF export will be used.")
 else:
 try:
 writer = imageio.get_writer(candidate_path, fps=RENDER_FPS, codec="libx264",
 macro_block_size=None)

```

```

 for frame in frames:
 writer.append_data(frame)
 writer.close()
 mp4_path = candidate_path
 print(f"Saved rendered video to: {mp4_path}")
 except Exception as exc:
 print(f"Could not save MP4 render. Falling back to GIF. Reason: {exc}")

```

```

if SAVE_RENDER_GIF:
 gif_path = output_dir / "painting_demo.gif"
 imageio.mimsave(gif_path, frames, fps=RENDER_FPS)
 print(f"Saved rendered GIF to: {gif_path}")
 if IPyImage is not None and display is not None:
 display(IPyImage(filename=str(gif_path)))

return mp4_path, gif_path

```

```

def wall_polygon(surface_x):
 z_min = PAINT_Z_CENTER - WALL_HALF_HEIGHT_Z
 z_max = PAINT_Z_CENTER + WALL_HALF_HEIGHT_Z
 y_min = -WALL_HALF_WIDTH_Y
 y_max = WALL_HALF_WIDTH_Y
 return np.array(
 [
 [surface_x, y_min, z_min],
 [surface_x, y_max, z_min],
 [surface_x, y_max, z_max],
 [surface_x, y_min, z_max],
],
 dtype=float,
)

```

```

def save_robot_3d_animation(log, output_dir):
 """Save a matplotlib 3D animation that shows the robot painting the wall."""
 if not SAVE_ROBOT_3D_ANIMATION or not log["robot_chain_points"]:
 return None, None

 sample_indices = list(range(0, len(log["time"]), ROBOT_ANIMATION_EVERY_N_STEPS))
 if sample_indices[-1] != len(log["time"]) - 1:
 sample_indices.append(len(log["time"]) - 1)

 robot_points = np.asarray(log["robot_chain_points"])
 ee_positions = np.column_stack([log["ee_x"], log["ee_y"], log["ee_z"]])
 time = np.asarray(log["time"])
 force_meas = np.asarray(log["force_meas"])

 fig = plt.figure(figsize=(8, 7))
 ax = fig.add_subplot(111, projection="3d")
 ax.set_title("3D robot animation while painting")
 ax.set_xlabel("x [m]")
 ax.set_ylabel("y [m]")
 ax.set_zlabel("z [m]")
 ax.set_xlim(0.0, 0.80)
 ax.set_ylim(-0.35, 0.35)
 ax.set_zlim(0.0, 0.70)
 ax.view_init(elev=ROBOT_ANIMATION_ELEVATION, azimuth=ROBOT_ANIMATION_AZIMUTH)
 ax.grid(True, alpha=0.3)

 robot_line, = ax.plot([], [], [], "-o", color="tab:blue", linewidth=3, markersize=5,
label="Robot")
 ee_trail_line, = ax.plot([], [], [], color="tab:red", linewidth=1.8, alpha=0.8, label="Brush
trail")
 brush_marker, = ax.plot([], [], [], "o", color="crimson", markersize=7, label="Brush tip")
 wall_artist = {"poly": None}
 status_text = ax.text2D(0.02, 0.95, "", transform=ax.transAxes)
 ax.legend(loc="upper right")

def init():

```

```

robot_line.set_data([], [])
robot_line.set_3d_properties([])
ee_trail_line.set_data([], [])
ee_trail_line.set_3d_properties([])
brush_marker.set_data([], [])
brush_marker.set_3d_properties([])
status_text.set_text("")
return robot_line, ee_trail_line, brush_marker, status_text

```

```

def update(frame_number):
 idx = sample_indices[frame_number]
 points = robot_points[idx]
 trail = ee_positions[: idx + 1]

 robot_line.set_data(points[:, 0], points[:, 1])
 robot_line.set_3d_properties(points[:, 2])

 ee_trail_line.set_data(trail[:, 0], trail[:, 1])
 ee_trail_line.set_3d_properties(trail[:, 2])

 brush_marker.set_data([points[-1, 0]], [points[-1, 1]])
 brush_marker.set_3d_properties([points[-1, 2]])

 if wall_artist["poly"] is not None:
 wall_artist["poly"].remove()
 wall_vertices = [wall_polygon(log["wall_surface_x"][idx])]
 wall_artist["poly"] = Poly3DCollection(
 wall_vertices,
 facecolors=(0.55, 0.75, 0.95, 0.28),
 edgecolors=(0.25, 0.45, 0.75, 0.9),
 linewidths=1.0,
)
 ax.add_collection3d(wall_artist["poly"])

 status_text.set_text(
 f"t = {time[idx]:.2f} s\n"
 f"F = {force_meas[idx]:.2f} N\n"
 f"x_cmd = {log['x_cmd'][idx]:.3f} m"
)
 return robot_line, ee_trail_line, brush_marker, status_text, wall_artist["poly"]

```

```

animation_object = animation.FuncAnimation(
 fig,
 update,
 init_func=init,
 frames=len(sample_indices),
 interval=1000 / ROBOT_ANIMATION_FPS,
 blit=False,
)

```

```

gif_path = output_dir / "robot_painting_3d.gif"
animation_object.save(gif_path, writer=animation.PillowWriter(fps=ROBOT_ANIMATION_FPS), dpi=120)
plt.close(fig)

```

```

print(f"Saved matplotlib 3D animation to: {gif_path}")
if IPyImage is not None and display is not None:
 display(IPyImage(filename=str(gif_path)))

```

```

mp4_path = None
if imageio_ffmpeg is not None:
 try:

```

```

 mp4_path = output_dir / "robot_painting_3d.mp4"
 writer = imageio.get_writer(mp4_path, fps=ROBOT_ANIMATION_FPS, codec="libx264",
macro_block_size=None)
 reader = imageio.get_reader(gif_path)
 for frame in reader:
 writer.append_data(np.asarray(frame))
 reader.close()
 writer.close()
 print(f"Saved matplotlib 3D MP4 to: {mp4_path}")

```

```
except Exception as exc:
 print(f"Could not convert the 3D GIF into MP4. Reason: {exc}")
```

```
return gif_path, mp4_path
```

```
def get_output_dir():
 return choose_writable_dir(Script_DIR / "mujoco_painting_outputs", "mujoco_painting_outputs")
```

```
%% Main simulation
```

```
def run_single_case(
 case_name="single",
 wall_retreat_distance=None,
 wall_profile_mode=None,
 concave_depth=None,
 concave_sigma=None,
 random_wall_x=False,
 wall_x_min=0.64,
 wall_x_max=0.69,
 fixed_wall_x_center=None,
 unknown_wall_preset="stable",
 wall_motion_enabled=True,
):
 global WALL_RETREAT_DISTANCE, WALL_PROFILE_MODE, CONCAVE_DEPTH, CONCAVE_SIGMA, WALL_X_CENTER,
 WALL_MOTION_ENABLED
 original_wall_retreat_distance = WALL_RETREAT_DISTANCE
 original_wall_profile_mode = WALL_PROFILE_MODE
 original_concave_depth = CONCAVE_DEPTH
 original_concave_sigma = CONCAVE_SIGMA
 original_wall_x_center = WALL_X_CENTER
 original_wall_motion_enabled = WALL_MOTION_ENABLED

 if wall_retreat_distance is not None:
 WALL_RETREAT_DISTANCE = float(wall_retreat_distance)
 if wall_profile_mode is not None:
 WALL_PROFILE_MODE = str(wall_profile_mode)
 if concave_depth is not None:
 CONCAVE_DEPTH = float(concave_depth)
 if concave_sigma is not None:
 CONCAVE_SIGMA = float(concave_sigma)
 if fixed_wall_x_center is not None:
 WALL_X_CENTER = float(fixed_wall_x_center)
 elif random_wall_x:
 WALL_X_CENTER = float(np.random.uniform(wall_x_min, wall_x_max))
 WALL_MOTION_ENABLED = bool(wall_motion_enabled)

 control_mode = "unknown_wall"
 unknown_params = UNKNOWN_WALL_PRESETS.get(unknown_wall_preset,
 UNKNOWN_WALL_PRESETS["stable"]).copy()
 contact_threshold = float(unknown_params["contact_threshold"])
 contact_lost_threshold = float(unknown_params["contact_lost_threshold"])
 f_max_safe = float(unknown_params["f_max_safe"])
 v_search = float(unknown_params["v_search"])
 v_recovery = float(unknown_params["v_recovery"])
 k_force = float(unknown_params["k_force"])
 max_dx_per_step = float(unknown_params["max_dx_per_step"])
 max_approach_distance = float(unknown_params["max_approach_distance"])
 max_recovery_distance = float(unknown_params["max_recovery_distance"])
 contact_confirm_steps = int(unknown_params["contact_confirm_steps"])
 contact_lost_confirm_steps = int(unknown_params["contact_lost_confirm_steps"])
 unknown_xcmd_lpf_alpha = float(unknown_params["unknown_xcmd_lpf_alpha"])

 print(f"Python interpreter: {sys.executable}")
 print(f"Script directory: {Script_DIR}")
 print(
 f"Running case: {case_name} | mode={control_mode} | wall_profile={WALL_PROFILE_MODE} "
 f"| wall_motion={'moving' if WALL_MOTION_ENABLED else 'static'} | WALL_X_CENTER="
 f"{WALL_X_CENTER:.4f} "
 f"| unknown_wall_preset={unknown_wall_preset}"
)
```

```

)

model, model_source = load_model()
data = mujoco.MjData(model)
ik_data = mujoco.MjData(model)

site_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, EE_SITE_NAME)
if site_id < 0:
 raise ValueError(
 f"Site '{EE_SITE_NAME}' was not found. Update EE_SITE_NAME after swapping in a real robot
model."
)

wall_body_id = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, WALL_BODY_NAME)
if wall_body_id < 0:
 raise ValueError(
 f"Wall body '{WALL_BODY_NAME}' was not found. Keep the moving wall name consistent with
this script."
)
mocap_id = model.body_mocapid[wall_body_id]
if mocap_id < 0:
 raise ValueError("The wall body is expected to be a mocap body in this demo.")

paint_mark_site_ids = get_paint_mark_site_ids(model)
reset_paint_marks(model, paint_mark_site_ids)

qpos_indices, dof_indices, joint_lower, joint_upper = get_joint_indices(model, JOINT_NAMES)
actuator_indices = get_joint_actuator_indices(model, JOINT_NAMES)
robot_chain_body_ids = get_robot_chain_body_ids(model, JOINT_NAMES)

Initialize the wall pose.
initial_wall_x, _ = wall_motion(0.0)
data.mocap_pos[mocap_id] = np.array([initial_wall_x, 0.0, PAINT_Z_CENTER])
data.mocap_quat[mocap_id] = np.array([1.0, 0.0, 0.0, 0.0])
ik_data.mocap_pos[mocap_id] = data.mocap_pos[mocap_id].copy()
ik_data.mocap_quat[mocap_id] = data.mocap_quat[mocap_id].copy()

Use IK to find a reasonable starting joint configuration at the initial painting pose.
q_seed = np.zeros(len(JOINT_NAMES))
initial_target, _ = painting_nominal_pose(0.0)
initial_target[0] = X_NOMINAL_BASE - PEN_UP_CLEARANCE
q_init = ik_damped_least_squares(
 model,
 ik_data,
 site_id,
 initial_target,
 q_seed,
 qpos_indices,
 dof_indices,
 joint_lower,
 joint_upper,
)

data.qpos[qpos_indices] = q_init
data.qvel[:] = 0.0
data.ctrl[:] = 0.0
data.ctrl[actuator_indices] = q_init
mujoco.mj_forward(model, data)

ik_data.qpos[qpos_indices] = q_init
ik_data.qvel[:] = 0.0
ik_data.ctrl[:] = 0.0
ik_data.ctrl[actuator_indices] = q_init
mujoco.mj_forward(model, ik_data)

renderer = None
camera = None
scene_option = None
frames = []
viewer_handle = maybe_launch_native_viewer(model, data)

```



```

if should_attempt_rendering():
 try:
 renderer, camera, scene_option = make_renderer(model)
 except Exception as exc:
 print(f"Renderer could not be created. Continuing without video. Reason: {exc}")

log = {
 "control_mode": control_mode,
 "time": [],
 "wall_x": [],
 "wall_surface_x": [],
 "true_wall_surface_x": [],
 "ee_x": [],
 "ee_y": [],
 "ee_z": [],
 "x_cmd_raw": [],
 "x_cmd": [],
 "force_meas": [],
 "force_manual": [],
 "force_touch": [],
 "force_des": [],
 "force_error": [],
 "controller_state": [],
 "contact_confirm_count": [],
 "contact_lost_count": [],
 "x_search_start": [],
 "x_recovery_start": [],
 "x_contact_est": [],
 "joint_positions": [],
 "joint_commands": [],
 "contact_lost": [],
 "robot_chain_points": [],
 "pen_down": [],
 "penetration": [],
}

q_command = q_init.copy()
sim_duration_budget = effective_sim_duration()
if control_mode == "unknown_wall":
 sim_duration_budget *= UNKNOWN_WALL_PATH_TIME_BUDGET_SCALE
num_steps = int(sim_duration_budget / DT)
next_paint_mark_index = 0
last_paint_mark_local_yz = None
previous_pen_down = False
pen_down_elapsed = 0.0
controller_state = "APPROACH"
contact_confirm_count = 0
contact_lost_count = 0
x_search_start = None
x_recovery_start = None
x_contact_est = None
unknown_safety_warned = False
yz_hold = None
path_time = 0.0
path_time_can_advance = True
initial_ee_x = float(get_ee_pose(model, data, site_id)[0])
x_unknown_min = initial_ee_x - 0.03
x_unknown_max = initial_ee_x + 0.08

try:
 for step in range(num_steps):
 if viewer_handle is not None and not viewer_handle.is_running():
 print("Viewer window was closed early; ending simulation loop.")
 break

 step_wallclock_start = time.time()
 t = step * DT

 wall_x, _ = wall_motion(t)
 data.mocap_pos[mocap_id] = np.array([wall_x, 0.0, PAINT_Z_CENTER])

```

```
data.mocap_quat[mocap_id] = np.array([1.0, 0.0, 0.0, 0.0])
mujoco.mj_forward(model, data)
```

```
contact = contact_force(model, data, site_id, t)
if control_mode == "unknown_wall":
 if step > 0 and path_time_can_advance:
 path_time += DT
 path_time = min(path_time, effective_sim_duration())
 p_nominal, pen_down = painting_nominal_pose(path_time)
else:
 p_nominal, pen_down = painting_nominal_pose(t)
yz_tracking_enabled = True

if pen_down and not previous_pen_down:
 pen_down_elapsed = 0.0
 controller_state = "APPROACH"
 contact_confirm_count = 0
 contact_lost_count = 0
 x_search_start = None
 x_recovery_start = None
 ee_pos_now = get_ee_pose(model, data, site_id)
 yz_hold = np.array([float(ee_pos_now[1]), float(ee_pos_now[2])], dtype=float)

if pen_down:
 force_measured = contact["force_used"]
 force_desired = F_DES
 previous_x_cmd = log["x_cmd"][-1] if len(log["x_cmd"]) > 0 else float(contact["ee_x"])
 ee_vx = float(get_site_linear_velocity(model, data, site_id)[0])

 ee_pos = get_ee_pose(model, data, site_id)
 x_cmd_raw = previous_x_cmd
 force_error = F_DES - force_measured

 if force_measured > f_max_safe:
 controller_state = "SAFETY_STOP"

 if controller_state == "APPROACH":
 if x_search_start is None:
 x_search_start = previous_x_cmd
 x_cmd_raw = previous_x_cmd + v_search * DT
 if (x_cmd_raw - x_search_start) > max_approach_distance:
 controller_state = "SAFETY_STOP"
 x_cmd_raw = previous_x_cmd
 if force_measured > contact_threshold:
 contact_confirm_count += 1
 else:
 contact_confirm_count = 0
 if contact_confirm_count >= contact_confirm_steps:
 controller_state = "PAINTING"
 x_contact_est = float(ee_pos[0])
 x_cmd_raw = previous_x_cmd
 contact_lost_count = 0
 elif controller_state == "PAINTING":
 dx = k_force * force_error * DT
 dx = float(np.clip(dx, -max_dx_per_step, max_dx_per_step))
 x_cmd_raw = previous_x_cmd + dx
 if force_measured < contact_lost_threshold:
 contact_lost_count += 1
 else:
 contact_lost_count = 0
 if contact_lost_count >= contact_lost_confirm_steps:
 controller_state = "RECOVERY"
 x_recovery_start = previous_x_cmd
 elif controller_state == "RECOVERY":
 x_cmd_raw = previous_x_cmd + v_recovery * DT
 if x_recovery_start is None:
 x_recovery_start = previous_x_cmd
 if (x_cmd_raw - x_recovery_start) > max_recovery_distance:
 controller_state = "SAFETY_STOP"
 x_cmd_raw = previous_x_cmd
```

```

 if force_measured > contact_threshold:
 contact_confirm_count += 1
 else:
 contact_confirm_count = 0
 if contact_confirm_count >= contact_confirm_steps:
 controller_state = "PAINTING"
 x_contact_est = float(ee_pos[0])
 x_cmd_raw = previous_x_cmd
 contact_lost_count = 0
 else:
 x_cmd_raw = previous_x_cmd
 if not unknown_safety_warned:
 print("Safety stop: wall not found, contact lost too long, or force exceeded
limit.")

 unknown_safety_warned = True

x_cmd_target = float(np.clip(x_cmd_raw, x_unknown_min, x_unknown_max))
max_step = UNKNOWN_MAX_XCMD_RATE * DT
x_cmd_rate_limited = float(
 np.clip(x_cmd_target, previous_x_cmd - max_step, previous_x_cmd + max_step)
)
x_cmd = float(
 (1.0 - unknown_xcmd_lpf_alpha) * previous_x_cmd
 + unknown_xcmd_lpf_alpha * x_cmd_rate_limited
)
x_cmd = float(np.clip(x_cmd, x_unknown_min, x_unknown_max))
in_contact = force_measured > contact_threshold
yz_tracking_enabled = (controller_state == "PAINTING") and in_contact
if yz_tracking_enabled:
 yz_hold = np.array([float(p_nominal[1]), float(p_nominal[2])], dtype=float)
elif yz_hold is None:
 yz_hold = np.array([float(ee_pos[1]), float(ee_pos[2])], dtype=float)
else:
 force_measured = contact["force_used"]
 in_contact = False
 pen_down_elapsed = 0.0
 force_desired = 0.0
 previous_x_cmd = log["x_cmd"][-1] if len(log["x_cmd"]) > 0 else float(contact["ee_x"])
 x_cmd_raw = float(min(previous_x_cmd, X_NOMINAL_BASE - PEN_UP_CLEARANCE))
 x_cmd_target = float(np.clip(x_cmd_raw, x_unknown_min, x_unknown_max))
 max_step = UNKNOWN_MAX_XCMD_RATE * DT
 x_cmd_rate_limited = float(
 np.clip(x_cmd_target, previous_x_cmd - max_step, previous_x_cmd + max_step)
)
 x_cmd = float(
 (1.0 - unknown_xcmd_lpf_alpha) * previous_x_cmd
 + unknown_xcmd_lpf_alpha * x_cmd_rate_limited
)
 x_cmd = float(np.clip(x_cmd, x_unknown_min, x_unknown_max))
 controller_state = "APPROACH"
 contact_confirm_count = 0
 contact_lost_count = 0
 x_search_start = None
 x_recovery_start = None
 force_error = 0.0
 yz_tracking_enabled = False

path_time_can_advance = (not pen_down) or (controller_state in ("PAINTING", "RECOVERY"))

p_command = p_nominal.copy()
p_command[0] = x_cmd
if not yz_tracking_enabled and yz_hold is not None:
 p_command[1] = float(yz_hold[0])
 p_command[2] = float(yz_hold[1])

q_current = data.qpos[qpos_indices].copy()
q_command = ik_damped_least_squares(
 model,
 ik_data,
 site_id,

```

```

 p_command,
 q_current,
 qpos_indices,
 dof_indices,
 joint_lower,
 joint_upper,
)
data.ctrl[actuator_indices] = q_command

if USE_QUASI_STATIC_SERVO:
 data.qpos[qpos_indices] = q_command
 data.qvel[dof_indices] = 0.0
 mujoco.mj_forward(model, data)

ee_pos = get_ee_pose(model, data, site_id)
robot_chain_points = get_robot_chain_points(model, data, robot_chain_body_ids, site_id)

paint_allowed = (
 pen_down
 and in_contact
 and controller_state == "PAINTING"
 and controller_state != "SAFETY_STOP"
 and force_measured > contact_threshold
)
if paint_allowed:
 next_paint_mark_index, last_paint_mark_local_yz = maybe_add_paint_mark(
 model,
 paint_mark_site_ids,
 next_paint_mark_index,
 ee_pos,
 last_paint_mark_local_yz,
)

log["time"].append(t)
log["wall_x"].append(contact["wall_x"])
log["wall_surface_x"].append(contact["wall_surface_x"])
log["true_wall_surface_x"].append(contact["wall_surface_x"])
log["ee_x"].append(ee_pos[0])
log["ee_y"].append(ee_pos[1])
log["ee_z"].append(ee_pos[2])
log["x_cmd_raw"].append(x_cmd_raw)
log["x_cmd"].append(x_cmd)
log["force_meas"].append(force_measured)
log["force_manual"].append(contact["force_manual"])
log["force_touch"].append(contact["force_touch"])
log["force_des"].append(force_desired)
log["force_error"].append(force_error)
log["controller_state"].append(controller_state)
log["contact_confirm_count"].append(contact_confirm_count)
log["contact_lost_count"].append(contact_lost_count)
log["x_search_start"].append(np.nan if x_search_start is None else float(x_search_start))
log["x_recovery_start"].append(np.nan if x_recovery_start is None else
float(x_recovery_start))
log["x_contact_est"].append(np.nan if x_contact_est is None else float(x_contact_est))
log["joint_positions"].append(q_current.copy())
log["joint_commands"].append(q_command.copy())
log["contact_lost"].append(0 if in_contact else 1)
log["robot_chain_points"].append(robot_chain_points)
log["pen_down"].append(1 if pen_down else 0)
log["penetration"].append(contact["penetration"])
if pen_down:
 pen_down_elapsed += DT
previous_pen_down = pen_down

if renderer is not None and step % RENDER_EVERY_N_STEPS == 0:
 renderer.update_scene(data, camera=camera, scene_option=scene_option)
 pixels = renderer.render()
 frames.append(np.asarray(pixels).copy())

if viewer_handle is not None:

```

```
viewer_handle.sync()
```

```
mujoco.mj_step(model, data)
```

```
if viewer_handle is not None and VIEWER_REALTIME:
 elapsed = time.time() - step_wallclock_start
 remaining = DT - elapsed
 if remaining > 0:
 time.sleep(remaining)
```

```
if viewer_handle is not None and KEEP_VIEWER_OPEN_AFTER_SIM and viewer_handle.is_running():
 print("Simulation finished. Close the MuJoCo viewer window when you are done inspecting
```

```
it.")
```

```
while viewer_handle.is_running():
 viewer_handle.sync()
 time.sleep(0.02)
```

```
finally:
```

```
 WALL_RETREAT_DISTANCE = original_wall_retreat_distance
 WALL_PROFILE_MODE = original_wall_profile_mode
 CONCAVE_DEPTH = original_concave_depth
 CONCAVE_SIGMA = original_concave_sigma
 WALL_X_CENTER = original_wall_x_center
 WALL_MOTION_ENABLED = original_wall_motion_enabled
 if renderer is not None:
 renderer.close()
 if viewer_handle is not None:
 viewer_handle.close()
```

```
output_dir = get_output_dir() / case_name
output_dir.mkdir(parents=True, exist_ok=True)
```

```
fig_paths = plot_results(log, output_dir)
mp4_path, gif_path = save_render_outputs(frames, output_dir) if frames else (None, None)
robot_animation_gif_path, robot_animation_mp4_path = save_robot_3d_animation(log, output_dir)
```

```
sim_time = np.asarray(log["time"])
wall_x_log = np.asarray(log["wall_x"])
force_meas = np.asarray(log["force_meas"])
x_cmd_log = np.asarray(log["x_cmd"])
ee_x_log = np.asarray(log["ee_x"])
pen_down_mask = np.asarray(log["pen_down"], dtype=bool)
pen_down_force = force_meas[pen_down_mask]
painting_mask = pen_down_mask
if control_mode == "unknown_wall":
 painting_mask = pen_down_mask & (np.asarray(log["controller_state"]) == "PAINTING")
pen_down_contact_lost = np.asarray(log["contact_lost"])[pen_down_mask]
contact_loss_fraction = float(np.mean(pen_down_contact_lost)) if pen_down_contact_lost.size > 0
else float("nan")
mean_force_pen_down = float(np.mean(pen_down_force)) if pen_down_force.size > 0 else float("nan")
max_force_pen_down = float(np.max(pen_down_force)) if pen_down_force.size > 0 else float("nan")
mean_abs_force_error_pen_down = (
 float(np.mean(np.abs(force_meas[painting_mask] - F_DES))) if np.any(painting_mask) else
 float("nan")
)
min_force_pen_down = float(np.min(force_meas[painting_mask])) if np.any(painting_mask) else
float("nan")
max_abs_cmd_correction = float(np.max(np.abs(x_cmd_log - ee_x_log))) if x_cmd_log.size > 0 else
float("nan")
```

```
longest_loss_steps = 0
current_loss_steps = 0
for pd, lost in zip(log["pen_down"], log["contact_lost"]):
 if pd and lost:
 current_loss_steps += 1
 longest_loss_steps = max(longest_loss_steps, current_loss_steps)
```

```
 elif pd:
 current_loss_steps = 0
```

```
longest_contact_loss_duration = longest_loss_steps * DT
safety_stop_count = int(np.sum(np.asarray(log["controller_state"]) == "SAFETY_STOP"))
recovery_events = 0
```

```

previous_state = None
for s in log["controller_state"]:
 if s == "RECOVERY" and previous_state != "RECOVERY":
 recovery_events += 1
 previous_state = s
first_contact_indices = np.where((pen_down_mask) & (force_meas > contact_threshold))[0]
time_to_first_contact = float(sim_time[first_contact_indices[0]]) if first_contact_indices.size >
0 else float("nan")

```

```

print("\n=== Simulation summary ===")
print(f"Controller mode: {control_mode}")
print(f"Model source: {model_source}")
print(f"Simulated time: {sim_time[-1]:.2f} s")
print(f"Wall x range: [{np.min(wall_x_log):.3f}, {np.max(wall_x_log):.3f}] m")
print(f"Mean measured force (pen-down): {mean_force_pen_down:.2f} N")
print(f"Max measured force (pen-down): {max_force_pen_down:.2f} N")
print(f"Mean |force error| (pen-down): {mean_abs_force_error_pen_down:.2f} N")
print(f"Minimum measured force (pen-down): {min_force_pen_down:.2f} N")
print(f"Contact-lost fraction (pen-down): {100.0 * contact_loss_fraction:.1f}%")
print(f"Longest continuous contact-loss (pen-down): {longest_contact_loss_duration:.3f} s")
print(f"Number of safety stops: {safety_stop_count}")
print(f"Time to first contact: {time_to_first_contact:.3f} s")
print(f"Number of recovery events: {recovery_events}")
print(f"Max |x_cmd - ee_x| commanded correction: {max_abs_cmd_correction:.4f} m")
print(f"Plots saved to: {output_dir}")
print(f'Painted text: "{PAINT_TEXT}"')

```

```

if mp4_path is not None:
 print(f"Rendered MP4 saved to: {mp4_path}")
if gif_path is not None:
 print(f"Rendered GIF saved to: {gif_path}")
if robot_animation_gif_path is not None:
 print(f"3D robot animation GIF saved to: {robot_animation_gif_path}")
if robot_animation_mp4_path is not None:
 print(f"3D robot animation MP4 saved to: {robot_animation_mp4_path}")

```

```

print("\nInterpretation:")

```

```

print("- This demo uses a one-sided wall motion: the wall first advances toward the robot, then
relaxes back while staying no farther than its starting x position.")
print("- When the wall moves toward the robot, the brush-wall penetration increases, so the
measured contact force rises.")
print("- The selected controller adjusts commanded x to regulate normal contact while painting.")
print("- Unknown-wall mode does not use true wall retreat or surface x for x_cmd control.")
print("- It uses only measured force and EE state to approach, paint, recover, and safety-stop.")

```

```

print("\nSaved figures:")

```

```

for path in fig_paths:
 print(f"- {path}")

```

```

return {
 "case_name": case_name,
 "control_mode": control_mode,
 "contact_lost_pct": 100.0 * contact_loss_fraction,
 "longest_contact_loss_s": longest_contact_loss_duration,
 "mean_force_error_pen_down": mean_abs_force_error_pen_down,
 "min_force_pen_down": min_force_pen_down,
 "safety_stops": safety_stop_count,
 "time_to_first_contact_s": time_to_first_contact,
 "recovery_events": recovery_events,
 "output_dir": str(output_dir),
}

```

```

def parse_cli_args():
 parser = argparse.ArgumentParser(description="MuJoCo painting demo with unknown-wall force
control.")
 parser.add_argument("--case-name", type=str, default="single")
 parser.add_argument(
 "--scenario",
 type=str,
 default="all",
 choices=["all", "flat_static", "bumpy_static", "flat_moving", "bumpy_moving"],
)

```

```

 help="Choose one scenario or run all four.",
)
 parser.add_argument("--random-wall-x", action="store_true", help="Randomize true wall x center for each run.")
 parser.add_argument("--wall-x-min", type=float, default=0.64)
 parser.add_argument("--wall-x-max", type=float, default=0.69)
 parser.add_argument(
 "--unknown-wall-preset",
 type=str,
 default="stable",
 choices=list(UNKNOWN_WALL_PRESETS.keys()),
 help="Preset for unknown-wall controller tuning.",
)
 args, unknown = parser.parse_known_args()
 if unknown:
 print(f"Ignoring unknown CLI args: {unknown}")
 return args

def run_four_scenarios(random_wall_x=False, wall_x_min=0.64, wall_x_max=0.69,
unknown_wall_preset="stable"):
 scenarios = [
 ("flat_static", "rigid", False),
 ("bumpy_static", "bumpy", False),
 ("flat_moving", "rigid", True),
 ("bumpy_moving", "bumpy", True),
]
 results = []
 for case_name, wall_profile_mode, wall_motion_enabled in scenarios:
 result = run_single_case(
 case_name=case_name,
 wall_profile_mode=wall_profile_mode,
 wall_motion_enabled=wall_motion_enabled,
 random_wall_x=bool(random_wall_x),
 wall_x_min=float(wall_x_min),
 wall_x_max=float(wall_x_max),
 unknown_wall_preset=str(unknown_wall_preset),
)
 results.append(result)

 print("\n=== Four Scenario Summary ===")
 for result in results:
 print(
 f"{result['case_name']}: lost={result['contact_lost_pct']:.2f}% | "
 f"mean|err|={result['mean_force_error_pen_down']:.3f} N | "
 f"minF={result['min_force_pen_down']:.3f} N | "
 f"safety_stops={result['safety_stops']} | out={result['output_dir']}"
)

%% Standalone 0524 task configuration and controller
from typing import List

def configure_task(base):
 # Use the existing Piper model + wall patch, and set a 15 cm brush.
 base.EXTERNAL_BRUSH_OFFSET = base.np.array([0.0, 0.0, 0.15], dtype=float)
 base.USE_TOUCH_SENSOR = False

 # Moving rigid wall. The MuJoCo mocap body is synchronized in the run loop.
 base.WALL_PROFILE_MODE = "rigid"
 base.WALL_MOTION_ENABLED = True
 base.WALL_PUSH_DISTANCE = 0.008
 base.WALL_RETREAT_DISTANCE = 0.006

 # Contact mechanics assumptions (brush + wall).
 base.CONTACT_STIFFNESS = 3000.0 # N/m
 base.CONTACT_DAMPING = 60.0 # N*s/m
 base.F_DES = 10.0 # N

 # UCLA painting path.
 base.PAINT_TEXT = "UCLA"

```

```

base.LETTER_WIDTH = 0.055
base.LETTER_HEIGHT = 0.095
base.LETTER_SPACING = 0.015
base.LETTER_STROKE_SPEED = 0.040
base.CA_STROKE_SPEED_SCALE = 0.50
base.PEN_UP_TRAVEL_SPEED = 0.18
base.PRE_STROKE_SETTLE_TIME = 0.20
base.CORNER_HOLD_C_TIME = 0.08
base.CORNER_HOLD_A_TIME = 0.08
base.SIM_DURATION = 20.0
base.PAINT_START_DELAY = 0.8
base.PAINT_END_HOLD = 1.0
Make text readable from the viewer/user side in your current camera setup.
base.TEXT_READABLE_FROM_ROBOT_SIDE = True
Keep base plan for compatibility; runtime uses a local strict plan in this file.
base.PAINT_PLAN = base.build_paint_plan()
Improve Cartesian tracking fidelity.
base.IK_MAX_ITERS = 60
base.IK_DAMPING = 1.5e-3

```

```

def build_local_ucla_plan(base):
 w = base.LETTER_WIDTH
 h = base.LETTER_HEIGHT
 s = base.LETTER_SPACING
 total_width = 4 * w + 3 * s
 x0 = -0.5 * total_width
 z_bottom = -0.5 * h
 z_top = 0.5 * h
 z_mid = 0.02 * h

 settle_before_stroke_s = float(getattr(base, "PRE_STROKE_SETTLE_TIME", 0.40))
 corner_hold_c_s = float(getattr(base, "CORNER_HOLD_C_TIME", 0.14))
 corner_hold_a_s = float(getattr(base, "CORNER_HOLD_A_TIME", 0.14))
 c_a_speed_scale = float(getattr(base, "CA_STROKE_SPEED_SCALE", 0.35))
 pen_up_clearance = float(getattr(base, "PEN_UP_Z_OFFSET", 0.03))
 base.PEN_UP_Z_OFFSET = pen_up_clearance

 def letter_origin(letter_index):
 return x0 + letter_index * (w + s)

 # Each stroke spec: points, speed_scale, optional corner_holds(after stroke end)
 letters: List[List[object]] = []

 u0 = letter_origin(0)
 letters.append(
 [
 {
 "points": base.numpy.array([[u0, z_top], [u0, z_bottom], [u0 + w, z_bottom], [u0 + w,
z_top]], dtype=float),
 "speed_scale": 1.0,
 "corner_holds": [],
 }
]
)

 c0 = letter_origin(1)
 letters.append(
 [
 {
 # C one-stroke path: top-right -> top-left -> down-left -> bottom-right
 "points": base.numpy.array(
 [
 [c0 + w, z_top],
 [c0 + 0.65 * w, z_top],
 [c0 + 0.30 * w, z_top],
 [c0, z_top],
 [c0, 0.50 * z_top],
 [c0, 0.0],
 [c0, 0.50 * z_bottom],
 [c0, z_bottom],
]
)
 }
]
)

```



```

 [c0 + 0.30 * w, z_bottom],
 [c0 + 0.65 * w, z_bottom],
 [c0 + w, z_bottom],
],
 dtype=float,
),
"speed_scale": c_a_speed_scale,
"corner_holds": [corner_hold_c_s, corner_hold_c_s],
},
]
)

l0 = letter_origin(2)
letters.append(
 [
 {
 "points": base.numpy.array([[l0, z_top], [l0, z_bottom], [l0 + w, z_bottom]]),
dtype=float),
 "speed_scale": 1.0,
 "corner_holds": [],
 }
]
)

a0 = letter_origin(3)
letters.append(
 [
 {
 # A one-stroke path: left leg up -> right leg down -> move to crossbar and draw
 "points": base.numpy.array(
 [
 [a0, z_bottom],
 [a0 + 0.25 * w, 0.5 * z_top],
 [a0 + 0.5 * w, z_top],
 [a0 + 0.75 * w, 0.5 * z_top],
 [a0 + w, z_bottom],
 [a0 + 0.75 * w, z_mid],
 [a0 + 0.25 * w, z_mid],
]
),
 dtype=float,
 },
 {
 "speed_scale": c_a_speed_scale,
 "corner_holds": [corner_hold_a_s],
 }
]
)

segments = []
cursor = None
for letter_idx, letter_strokes in enumerate(letters):
 first_stroke = letter_strokes[0]["points"]
 if cursor is not None and base.numpy.linalg.norm(first_stroke[0] - cursor) > 1e-9:
 safe_z = max(float(cursor[1]), float(first_stroke[0, 1])) + pen_up_clearance
 lift_point = base.numpy.array([cursor[0], safe_z], dtype=float)
 travel_point = base.numpy.array([first_stroke[0, 0], safe_z], dtype=float)
 descend_point = first_stroke[0].copy()

 segments.append({"points": base.numpy.vstack([cursor, lift_point]), "pen_down": False,
"speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
 segments.append({"points": base.numpy.vstack([lift_point, travel_point]), "pen_down": False,
"speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
 segments.append({"points": base.numpy.vstack([travel_point, descend_point]), "pen_down":
False, "speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})
 cursor = descend_point.copy()

 for stroke_spec in letter_strokes:
 stroke = stroke_spec["points"]
 stroke_speed = base.LETTER_STROKE_SPEED * float(stroke_spec.get("speed_scale", 1.0))
 if cursor is not None and base.numpy.linalg.norm(stroke[0] - cursor) > 1e-9:
 segments.append({"points": base.numpy.vstack([cursor, stroke[0]]), "pen_down": False,

```

```

speed": base.PEN_UP_TRAVEL_SPEED, "letter_idx": letter_idx})

Pre-stroke settle hold (pen-up) so controller can converge before contact.
segments.append(
 {
 "points": base.np.vstack([stroke[0], stroke[0]]),
 "pen_down": False,
 "speed": base.PEN_UP_TRAVEL_SPEED,
 "duration_override": settle_before_stroke_s,
 "letter_idx": letter_idx,
 }
)

Actual writing segment.
segments.append({"points": stroke, "pen_down": True, "speed": stroke_speed, "letter_idx":
letter_idx})

Optional short corner hold(s) for sharp turns.
for hold_dt in stroke_spec.get("corner_holds", []):
 segments.append(
 {
 "points": base.np.vstack([stroke[-1], stroke[-1]]),
 "pen_down": True,
 "speed": stroke_speed,
 "duration_override": float(hold_dt),
 "letter_idx": letter_idx,
 }
)
 cursor = stroke[-1].copy()

cumulative_time = base.PAINT_START_DELAY
timed = []
for segment in segments:
 length = base.polyline_length(segment["points"])
 if "duration_override" in segment:
 duration = float(segment["duration_override"])
 else:
 duration = 0.0 if length < 1e-9 else length / segment["speed"]
 timed.append(
 {
 "points": segment["points"],
 "pen_down": segment["pen_down"],
 "speed": segment["speed"],
 "letter_idx": int(segment.get("letter_idx", -1)),
 "t_start": cumulative_time,
 "t_end": cumulative_time + duration,
 "length": length,
 }
)
 cumulative_time += duration

def local_to_world(local_yz):
 y = base.PAINT_Y_CENTER - local_yz[0] if base.TEXT_READABLE_FROM_ROBOT_SIDE else
base.PAINT_Y_CENTER + local_yz[0]
 return base.np.array([base.X_NOMINAL_BASE, y, base.PAINT_Z_CENTER + local_yz[1]], dtype=float)

total_duration = cumulative_time + base.PAINT_END_HOLD
first_local_point = letters[0][0]["points"][0]
last_local_point = letters[-1][-1]["points"][-1]
local_plan = {
 "segments": timed,
 "total_duration": total_duration,
 "start_point": local_to_world(first_local_point),
 "end_point": local_to_world(last_local_point),
}
if bool(getattr(base, "PLAN_DEBUG_PRINT", True)):
 print("\n=== Local UCLA Plan Segments ===")
 for i, seg in enumerate(local_plan["segments"]):
 p0 = seg["points"][0]
 p1 = seg["points"][-1]

```

```

 print(
 f"[{i:02d}] letter={seg['letter_idx']} pen_down={seg['pen_down']} "
 f"t=[{seg['t_start']:.3f}, {seg['t_end']:.3f}] "
 f"start=({p0[0]:.4f},{p0[1]:.4f}) end=({p1[0]:.4f},{p1[1]:.4f})"
)
 return local_plan

def painting_nominal_pose_local(base, plan, t):
 if t <= base.PAINT_START_DELAY:
 return plan["start_point"].copy(), False, -1

 for segment in plan["segments"]:
 if segment["t_start"] <= t <= segment["t_end"] or (abs(t - segment["t_end"]) < 1e-9 and
segment["t_end"] >= segment["t_start"]):
 distance = segment["speed"] * max(0.0, t - segment["t_start"])
 local_point_yz = base.interpolate_polyline(segment["points"], distance)
 world_y = base.PAINT_Y_CENTER - local_point_yz[0] if base.TEXT_READABLE_FROM_ROBOT_SIDE
else base.PAINT_Y_CENTER + local_point_yz[0]
 return (
 base.numpy.array([base.X_NOMINAL_BASE, world_y, base.PAINT_Z_CENTER + local_point_yz[1]],
dtype=float),
 segment["pen_down"],
 int(segment.get("letter_idx", -1)),
)

 return plan["end_point"].copy(), False, -1

def tool_axis_error(base, current_rotation, target_tool_axis):
 current_tool_axis = current_rotation[:, 2]
 return base.numpy.cross(current_tool_axis, target_tool_axis)

def ik_pose_with_posture(
 base,
 model,
 ik_data,
 site_id,
 target_pos,
 target_tool_axis,
 q_seed,
 q_nominal,
 qpos_idx,
 dof_idx,
 joint_lower,
 joint_upper,
):
 q_trial = q_seed.copy()
 orientation_length_scale = 0.05
 posture_gain = 0.01
 posture_weights = base.numpy.array(
 [0.05, 0.05, 0.05, 0.25, 0.50, 1.00],
 dtype=float,
)

 for _ in range(base.IK_MAX_ITERS):
 ik_data.qpos[qpos_idx] = q_trial
 ik_data.qvel[:] = 0.0
 base.mujoco.mj_forward(model, ik_data)

 current_pos = ik_data.site_xpos[site_id].copy()
 current_rotation = ik_data.site_xmat[site_id].reshape(3, 3).copy()
 position_error = target_pos - current_pos
 orientation_error = tool_axis_error(
 base,
 current_rotation,
 target_tool_axis,
)
 if (
 base.numpy.linalg.norm(position_error) < base.IK_TOL
 and base.numpy.linalg.norm(orientation_error) < 2.0e-3
):

```

break

```
jacp = base.np.zeros((3, model.nv))
jacr = base.np.zeros((3, model.nv))
base.mujoco.mj_jacSite(model, ik_data, jacp, jacr, site_id)
jacobian = base.np.vstack(
 [
 jacp[:, dof_idx],
 orientation_length_scale * jacr[:, dof_idx],
]
)
task_error = base.np.concatenate(
 [
 position_error,
 orientation_length_scale * orientation_error,
]
)

regularized = (
 jacobian @ jacobian.T
 + (base.IK_DAMPING**2) * base.np.eye(6)
)
jacobian_pinv = jacobian.T @ base.np.linalg.solve(
 regularized,
 base.np.eye(6),
)
dq_primary = jacobian_pinv @ task_error
nullspace = base.np.eye(len(q_trial)) - jacobian_pinv @ jacobian
posture_step = posture_weights * (q_nominal - q_trial)
dq = dq_primary + posture_gain * (nullspace @ posture_step)
dq = base.np.clip(dq, -base.IK_STEP_LIMIT, base.IK_STEP_LIMIT)
q_trial = base.np.clip(q_trial + dq, joint_lower, joint_upper)
```

return q\_trial

```
def run_impedance_case(base):
 model, model_source = base.load_model()
 data = base.mujoco.MjData(model)
 ik_data = base.mujoco.MjData(model)

 site_id = base.mujoco.mj_name2id(model, base.mujoco.mjtObj.mjOBJ_SITE, base.EE_SITE_NAME)
 if site_id < 0:
 raise RuntimeError(f"Site '{base.EE_SITE_NAME}' not found in model.")

 wall_body_id = base.mujoco.mj_name2id(
 model,
 base.mujoco.mjtObj.mjOBJ_BODY,
 base.WALL_BODY_NAME,
)
 if wall_body_id < 0:
 raise RuntimeError(f"Wall body '{base.WALL_BODY_NAME}' not found in model.")
 wall_mocap_id = int(model.body_mocapid[wall_body_id])
 if wall_mocap_id < 0:
 raise RuntimeError("The wall body must be a MuJoCo mocap body.")

 initial_wall_x, _ = base.wall_motion(0.0)
 initial_wall_pos = base.np.array(
 [initial_wall_x, 0.0, base.PAINT_Z_CENTER],
 dtype=float,
)
 initial_wall_quat = base.np.array([1.0, 0.0, 0.0, 0.0], dtype=float)
 data.mocap_pos[wall_mocap_id] = initial_wall_pos
 data.mocap_quat[wall_mocap_id] = initial_wall_quat
 ik_data.mocap_pos[wall_mocap_id] = initial_wall_pos
 ik_data.mocap_quat[wall_mocap_id] = initial_wall_quat

 qpos_idx, dof_idx, joint_lower, joint_upper = base.get_joint_indices(model, base.JOINT_NAMES)
 act_idx = base.get_joint_actuator_indices(model, base.JOINT_NAMES)

 # Cartesian normal-direction impedance params (stabilized for position-actuated robot).
```

```

M_imp = 1.6
B_imp = 145.0
K_imp = 560.0
pen_up_clearance = 0.006
ex_limit = 0.015
v_limit = 0.12
x_cmd_rate_limit = 0.08 # m/s
force_lpf_alpha = 0.25
yz_track_kp = 0.65
yz_corr_limit = 0.020
yz_track_kp_c = 1.20
yz_corr_limit_c = 0.032
q_step_limit = base.numpy.array([0.03, 0.03, 0.03, 0.04, 0.04, 0.025], dtype=float)

Initial pose.
q_seed = base.numpy.zeros(len(base.JOINT_NAMES))
local_plan = build_local_ucla_plan(base)
p0, _, _ = painting_nominal_pose_local(base, local_plan, 0.0)
q_init = base.ik_damped_least_squares(
 model,
 ik_data,
 site_id,
 p0,
 q_seed,
 qpos_idx,
 dof_idx,
 joint_lower,
 joint_upper,
)
data.qpos[qpos_idx] = q_init
data.qvel[:] = 0.0
base.mujoco.mj_forward(model, data)
target_ee_rotation = data.site_xmat[site_id].reshape(3, 3).copy()
target_tool_axis = target_ee_rotation[:, 2].copy()
paint_mark_site_ids = base.get_paint_mark_site_ids(model)
base.reset_paint_marks(model, paint_mark_site_ids)
next_paint_mark_index = 0
last_paint_mark_local_yz = None
q_ref = q_init.copy()

sim_time = max(base.SIM_DURATION, float(local_plan["total_duration"]))
steps = int(sim_time / base.DT)
frames = []
renderer = None
camera = None
scene_option = None
if base.should_attempt_rendering():
 renderer, camera, scene_option = base.make_renderer(model)

x_imp = float(data.site_xpos[site_id][0])
x_imp_dot = 0.0
desired_penetration = base.F_DES / base.CONTACT_STIFFNESS
fixed_contact_equilibrium = (
 base.WALL_X_CENTER
 - base.WALL_HALF_THICKNESS
 - base.BRUSH_RADIUS
 + desired_penetration
)
x_ref_prev = x_imp
x_cmd_prev = x_imp
f_meas_filt = 0.0
log_t, log_f, log_fdes = [], [], []
log_xcmd, log_xee, log_xref = [], [], []
log_wall_surface, log_contact_gap = [], []
log_y_nom, log_z_nom, log_y_ee, log_z_ee = [], [], [], []
log_pen_down = []
log_orientation_error = []
log_q = []

for i in range(steps):

```

```

t = i * base.DT

wall_x, _ = base.wall_motion(t)
data.mocap_pos[wall_mocap_id] = base.numpy.array(
 [wall_x, 0.0, base.PAINT_Z_CENTER],
 dtype=float,
)
data.mocap_quat[wall_mocap_id] = initial_wall_quat
base.mujoco.mj_forward(model, data)

nominal_pos, pen_down, letter_idx = painting_nominal_pose_local(base, local_plan, t)
ee_pos = base.get_ee_pose(model, data, site_id)
contact = base.contact_force(model, data, site_id, t)

wall_surface_x = contact["wall_surface_x"]
x_ref = (
 fixed_contact_equilibrium
 if pen_down
 else fixed_contact_equilibrium - pen_up_clearance
)

f_des = base.F_DES if pen_down else 0.0
f_meas_raw = float(contact["force_used"])
if i == 0:
 f_meas_filt = f_meas_raw
else:
 f_meas_filt = force_lpf_alpha * f_meas_raw + (1.0 - force_lpf_alpha) * f_meas_filt
f_error = f_des - f_meas_filt

True impedance dynamics in wall-normal direction:
$M\ddot{e} + B\dot{e} + Ke = F_{des} - F_{meas}$
x_ref_dot = 0.0 if i == 0 else float((x_ref - x_ref_prev) / base.DT)
e = x_imp - x_ref
e_dot = x_imp_dot - x_ref_dot
e_ddot = (f_error - B_imp * e_dot - K_imp * e) / M_imp

x_imp_dot = float(base.numpy.clip(x_imp_dot + e_ddot * base.DT, -v_limit, v_limit))
x_imp = float(x_imp + x_imp_dot * base.DT)
x_imp = float(base.numpy.clip(x_imp, x_ref - ex_limit, x_ref + ex_limit))

x_cmd_target = x_imp
max_step = x_cmd_rate_limit * base.DT
x_cmd = float(base.numpy.clip(x_cmd_target, x_cmd_prev - max_step, x_cmd_prev + max_step))
x_ref_prev = x_ref
x_cmd_prev = x_cmd
target = nominal_pos.copy()
target[0] = x_cmd
Shape-first correction: keep y-z close to writing trajectory while x follows impedance.
yz_err = nominal_pos[1:3] - ee_pos[1:3]
if base.USE_QUASI_STATIC_SERVO:
 yz_corr = base.numpy.zeros(2, dtype=float)
elif pen_down and letter_idx == 1:
 yz_corr = base.numpy.clip(yz_track_kp_c * yz_err, -yz_corr_limit_c, yz_corr_limit_c)
else:
 yz_corr = base.numpy.clip(yz_track_kp * yz_err, -yz_corr_limit, yz_corr_limit)
target[1:3] = target[1:3] + yz_corr

q_curr = data.qpos[qpos_idx].copy()
q_cmd = ik_pose_with_posture(
 base,
 model,
 ik_data,
 site_id,
 target,
 target_tool_axis,
 q_curr,
 q_ref,
 qpos_idx,
 dof_idx,
 joint_lower,

```

```

 joint_upper,
)
 q_cmd = q_curr + base.np.clip(q_cmd - q_curr, -q_step_limit, q_step_limit)
 q_cmd[-1] = q_ref[-1]
 data.ctrl[act_idx] = q_cmd
 if base.USE_QUASI_STATIC_SERVO:
 data.qpos[qpos_idx] = q_cmd
 data.qvel[dof_idx] = 0.0
 base.mujoco.mj_forward(model, data)

 log_t.append(t)
 log_f.append(f_meas_filt)
 log_fdes.append(f_des)
 log_xcmd.append(x_cmd)
 log_xee.append(float(ee_pos[0]))
 log_xref.append(x_ref)
 log_wall_surface.append(float(wall_surface_x))
 log_contact_gap.append(
 float(wall_surface_x - (ee_pos[0] + base.BRUSH_RADIUS))
)
 log_y_nom.append(float(nominal_pos[1]))
 log_z_nom.append(float(nominal_pos[2]))
 log_y_ee.append(float(ee_pos[1]))
 log_z_ee.append(float(ee_pos[2]))
 log_pen_down.append(bool(pen_down))
 current_rotation = data.site_xmat[site_id].reshape(3, 3)
 log_orientation_error.append(
 float(base.np.linalg.norm(tool_axis_error(base, current_rotation, target_tool_axis)))
)
 log_q.append(data.qpos[qpos_idx].copy())

 if renderer is not None and (i % base.RENDER_EVERY_N_STEPS == 0):
 renderer.update_scene(data, camera=camera, scene_option=scene_option)
 pixels = renderer.render()
 frames.append(base.np.asarray(pixels).copy())

 base.mujoco.mj_step(model, data)
 if pen_down:
 ee_pos_mark = base.get_ee_pose(model, data, site_id)
 next_paint_mark_index, last_paint_mark_local_yz = base.maybe_add_paint_mark(
 model,
 paint_mark_site_ids,
 next_paint_mark_index,
 ee_pos_mark,
 last_paint_mark_local_yz,
)

out_dir = base.get_output_dir() / "0524_force_impedance"
out_dir.mkdir(parents=True, exist_ok=True)

base.plt.figure(figsize=(10, 7))
ax1 = base.plt.subplot(2, 1, 1)
ax1.plot(log_t, log_f, label="measured force")
ax1.plot(log_t, log_fdes, "--", label="desired force")
ax1.set_ylabel("Force [N]")
ax1.grid(True, alpha=0.3)
ax1.legend()

ax2 = base.plt.subplot(2, 1, 2)
ax2.plot(log_t, log_xcmd, label="x_cmd")
ax2.plot(log_t, log_xee, label="ee_x")
ax2.plot(log_t, log_xref, "--", label="fixed equilibrium")
ax2.plot(log_t, log_wall_surface, ":", label="wall surface")
ax2.set_xlabel("Time [s]")
ax2.set_ylabel("X [m]")
ax2.grid(True, alpha=0.3)
ax2.legend()
base.plt.tight_layout()
fig_path = out_dir / "impedance_response.png"
base.plt.savefig(fig_path, dpi=150)

```

```
base=plt.close()
```

```
retreat_start = base.WALL_MOTION_START + base.WALL_PUSH_DURATION
```

```
retreat_end = retreat_start + base.WALL_RETURN_DURATION
```

```
retreat_mask = (
 (base.numpy.asarray(log_t) >= retreat_start)
 & (base.numpy.asarray(log_t) <= retreat_end)
)
```

```
retreat_wall = base.numpy.asarray(log_wall_surface)[retreat_mask]
```

```
retreat_ee = base.numpy.asarray(log_xee)[retreat_mask]
```

```
retreat_force = base.numpy.asarray(log_f)[retreat_mask]
```

```
retreat_gap = base.numpy.asarray(log_contact_gap)[retreat_mask]
```

```
fig_follow = base.pyplot.figure(figsize=(10, 7))
```

```
follow_x = fig_follow.add_subplot(2, 1, 1)
```

```
follow_x.plot(log_t, log_wall_surface, label="wall surface")
```

```
follow_x.plot(
 log_t,
 base.numpy.asarray(log_xee) + base.BRUSH_RADIUS,
 label="brush front",
)
```

```
follow_x.set_ylabel("X [m]")
```

```
follow_x.grid(True, alpha=0.3)
```

```
follow_x.legend()
```

```
follow_gap = fig_follow.add_subplot(2, 1, 2)
```

```
follow_gap.plot(log_t, 1000.0 * base.numpy.asarray(log_contact_gap), label="contact gap")
```

```
follow_gap.axhline(0.0, color="black", linestyle="--", linewidth=1.0)
```

```
follow_gap.set_xlabel("Time [s]")
```

```
follow_gap.set_ylabel("Gap [mm]")
```

```
follow_gap.grid(True, alpha=0.3)
```

```
follow_gap.legend()
```

```
fig_follow.tight_layout()
```

```
follow_path = out_dir / "wall_retreat_following.png"
```

```
fig_follow.savefig(follow_path, dpi=160)
```

```
base.pyplot.close(fig_follow)
```

```
Trajectory accuracy diagnostics in wall tangential plane (y-z).
```

```
t_arr = base.numpy.asarray(log_t)
```

```
y_nom = base.numpy.asarray(log_y_nom)
```

```
z_nom = base.numpy.asarray(log_z_nom)
```

```
y_ee = base.numpy.asarray(log_y_ee)
```

```
z_ee = base.numpy.asarray(log_z_ee)
```

```
pen_down_arr = base.numpy.asarray(log_pen_down, dtype=bool)
```

```
yz_err = base.numpy.sqrt((y_ee - y_nom) ** 2 + (z_ee - z_nom) ** 2)
```

```
yz_err_mm = 1000.0 * yz_err
```

```
if base.numpy.any(pen_down_arr):
```

```
 yz_err_pen_mm = yz_err_mm[pen_down_arr]
```

```
 mean_err_mm = float(base.numpy.mean(yz_err_pen_mm))
```

```
 max_err_mm = float(base.numpy.max(yz_err_pen_mm))
```

```
else:
```

```
 mean_err_mm = float("nan")
```

```
 max_err_mm = float("nan")
```

```
fig_err = base.pyplot.figure(figsize=(11, 5))
```

```
ax_traj = fig_err.add_subplot(1, 2, 1)
```

```
ax_traj.plot(y_nom, z_nom, "--", linewidth=2.0, label="Nominal YZ path")
```

```
ax_traj.plot(y_ee, z_ee, linewidth=2.0, label="Actual YZ path")
```

```
ax_traj.set_xlabel("y [m]")
```

```
ax_traj.set_ylabel("z [m]")
```

```
ax_traj.set_title("Path Shape: Nominal vs Actual")
```

```
ax_traj.grid(True, alpha=0.3)
```

```
ax_traj.legend()
```

```
ax_traj.axis("equal")
```

```
ax_err = fig_err.add_subplot(1, 2, 2)
```

```
ax_err.plot(t_arr, yz_err_mm, linewidth=2.0, label="YZ tracking error")
```

```
if base.numpy.any(pen_down_arr):
```

```
 ax_err.plot(t_arr[pen_down_arr], yz_err_mm[pen_down_arr], ".", markersize=2.0, label="Pen-down
```



```

samples")
ax_err.set_xlabel("time [s]")
ax_err.set_ylabel("error [mm]")
ax_err.set_title("Tangential Tracking Error")
ax_err.grid(True, alpha=0.3)
ax_err.legend()
fig_err.tight_layout()
fig_err_path = out_dir / "yz_tracking_error.png"
fig_err.savefig(fig_err_path, dpi=160)
base.plt.close(fig_err)
mp4_path, gif_path = (None, None)
if frames:
 mp4_path, gif_path = base.save_render_outputs(frames, out_dir)
if renderer is not None:
 renderer.close()

print("=== True Impedance Control Run Complete ===")
print(f"Model source: {model_source}")
print("Controller: $M\ddot{e} + B\dot{e} + Ke = F_{des} - F_{meas}$ ")
print(f"Output dir: {out_dir}")
print(f"Figure: {fig_path}")
print(f"Wall following figure: {follow_path}")
print(f"YZ error figure: {fig_err_path}")
print(f"Mean YZ error (pen-down): {mean_err_mm:.2f} mm")
print(f"Max YZ error (pen-down): {max_err_mm:.2f} mm")
print(
 "Mean tool-axis drift: "
 f"{base.np.degrees(base.np.mean(log_orientation_error)):.2f} deg"
)
print(
 "Max tool-axis drift: "
 f"{base.np.degrees(base.np.max(log_orientation_error)):.2f} deg"
)
q_range_deg = base.np.degrees(base.np.ptp(base.np.asarray(log_q), axis=0))
print(
 "Wrist joint ranges (q4/q5/q6): "
 f"{q_range_deg[3]:.2f} / {q_range_deg[4]:.2f} / {q_range_deg[5]:.2f} deg"
)
metrics = {
 "planned_duration_s": float(local_plan["total_duration"]),
 "simulation_duration_s": float(sim_time),
 "video_fps": int(base.RENDER_FPS),
 "desired_force_n": float(base.F_DES),
 "mean_yz_error_pen_down_mm": mean_err_mm,
 "max_yz_error_pen_down_mm": max_err_mm,
 "mean_tool_axis_drift_deg": float(
 base.np.degrees(base.np.mean(log_orientation_error))
),
 "max_tool_axis_drift_deg": float(
 base.np.degrees(base.np.max(log_orientation_error))
),
 "wrist_joint_ranges_deg": {
 "q4": float(q_range_deg[3]),
 "q5": float(q_range_deg[4]),
 "q6": float(q_range_deg[5]),
 },
}
force_arr = base.np.asarray(log_f)
force_des_arr = base.np.asarray(log_fdes)
if base.np.any(pen_down_arr):
 force_error_pen = force_arr[pen_down_arr] - force_des_arr[pen_down_arr]
 metrics.update(
 {
 "mean_force_pen_down_n": float(base.np.mean(force_arr[pen_down_arr])),
 "force_mae_pen_down_n": float(base.np.mean(base.np.abs(force_error_pen))),
 "force_rmse_pen_down_n": float(
 base.np.sqrt(base.np.mean(force_error_pen**2))
),
 "force_std_pen_down_n": float(base.np.std(force_arr[pen_down_arr])),
 "peak_force_pen_down_n": float(base.np.max(force_arr[pen_down_arr])),
 }
)

```

```

 "minimum_force_pen_down_n": float(base.np.min(force_arr[pen_down_arr])),
 }
)

if retreat_wall.size > 1:
 wall_retreat_mm = 1000.0 * float(retreat_wall[-1] - retreat_wall[0])
 brush_follow_mm = 1000.0 * float(retreat_ee[-1] - retreat_ee[0])
 contact_ratio = 100.0 * float(base.np.mean(retreat_gap <= 0.0))
 mean_retreat_force = float(base.np.mean(retreat_force))
 follow_ratio = 100.0 * brush_follow_mm / wall_retreat_mm
 metrics.update(
 {
 "wall_retreat_mm": wall_retreat_mm,
 "brush_follow_mm": brush_follow_mm,
 "brush_follow_ratio_pct": follow_ratio,
 "retreat_contact_maintained_pct": contact_ratio,
 "mean_retreat_force_n": mean_retreat_force,
 "retreat_force_rmse_n": float(
 base.np.sqrt(base.np.mean((retreat_force - base.F_DES) ** 2))
),
 "maximum_retreat_gap_mm": float(1000.0 * base.np.max(retreat_gap)),
 "minimum_retreat_gap_mm": float(1000.0 * base.np.min(retreat_gap)),
 }
)
 print(f"Wall retreat distance: {wall_retreat_mm:.2f} mm")
 print(f"Brush follow distance: {brush_follow_mm:.2f} mm")
 print(f"Brush follow ratio: {follow_ratio:.1f}%")
 print(f"Retreat contact maintained: {contact_ratio:.1f}%")
 print(f"Mean retreat force: {mean_retreat_force:.2f} N")
summary_path = out_dir / "performance_summary.json"
summary_path.write_text(
 json.dumps(metrics, indent=2, ensure_ascii=False),
 encoding="utf-8",
)
print(f"Performance summary: {summary_path}")
if mp4_path is not None:
 print(f"MP4: {mp4_path}")
if gif_path is not None:
 print(f"GIF: {gif_path}")

def main():
 base = sys.modules[__name__]
 configure_task(base)
 run_impedance_case(base)

if __name__ == "__main__":
 main()

```